

SastraMap

Developer Onboarding & AI Handover Blueprint

An architecture analysis of `SastraMap_Done.html`: rendering pipeline, data dictionary, routing, live GPS and thermal governor, with an onboarding guide for developers and rules of engagement for AI agents. Version 1.1 adds the offline PWA shell and Campus Analytics layer.

Source file	SastraMap_Done.html — 4,500,857 bytes · 78,757 lines · 24 inline scripts
Engine	three.js r128 (inlined) · WebGL · no external assets · CSP connect-src none
Layout payload	v7 · buildStamp Bmuds6ios-9kbp · 290 placements · 41 host transforms
Build type	Dev build (World Editor mounts on desktops wider than 900 px)
Analysis date	27 September 2026 · document version 1.1 (adds Part IV)

PART I

Current Status & System Architecture

File anatomy, boot, WebGL pipeline, data dictionary, A* routers, Catmull-Rom splines, live GPS, governors.

PART II

Human Developer Onboarding Guide

Add a building, recalibrate X/Y/Z targets, extend the road graph, test UI safely, export and release.

PART III

Future AI Agent Rules of Engagement

Prime directives, memory-safety protocol, performance danger zones by line range, invariants, change protocol.

PART IV

PWA Offline Shell & Campus Analytics

Installable build, service worker, dead-zone behaviour, telemetry hooks, backend path, verification.

How to use this document

Part I describes the file as it is: its anatomy, boot, rendering pipeline, data layers, the two A* routers and their Catmull-Rom splines, the live GPS blue dot, and the environmental and thermal governors. **Part II** is a hands-on guide for developers: adding a building, recalibrating X/Y/Z targets, extending the road graph, and testing UI changes without stalling the render loop. **Part III** is the normative protocol that any AI agent working on the file must follow: memory-safety rules, the performance danger-zone map with line ranges, invariants and a change protocol. **Part IV** documents the installable, offline-capable build and its privacy-first telemetry, with the measured results of an end-to-end test.

Every figure here was read from the source file or measured from it. Counts marked "measured in code" repeat numbers the file's own comments record. Code listings in Part II (the payload round-trip, the road-graph validator and the syntax check) were run against this revision. **Line numbers drift:** search for the quoted banner title or function name instead.

Contents

Part I — Current Status & System Architecture	4
1. Executive summary	4
1.1 Status at a glance	4
2. File anatomy	5
3. Boot sequence	6
4. Coordinate system & the JSON Data Dictionary	6
4.1 Coordinate system	6
4.2 Layer 1 — Host survey tables (code, lines 2373–2700)	7
4.3 Layer 2 — The embedded layout payload (line 78757)	7
4.4 Layer 3 — The card data dictionary	8
4.5 The naming contract (critical)	8
5. The Three.js / WebGL rendering pipeline	8
5.1 Renderer configuration	8
5.2 Anatomy of one frame — animate() (line 11849)	9
5.3 Post-processing chain (renderFX, line 9906)	9
5.4 The program-stability model	9
5.5 CPU and draw-call savers	10
5.6 Scene flags every module respects	10
6. The glassmorphism UI layer	10
7. Pathfinding: A* routing and Catmull-Rom splines	11
7.1 Grid NavMesh A* in detail	11
7.2 Road-Graph A* in detail (AAA5 / AAA6)	12
7.3 Route memory model	12
8. Live GPS tracking (the Blue Dot)	13
9. Environmental systems & the thermal governor	13
9.1 Environment	13
9.2 The thermal governor, layer by layer	14

10. Observations & known issues	14
Part II — Human Developer Onboarding Guide	16
11. Setting up and daily workflow	16
12. Recipe A — Adding a new building	16
13. Recipe B — Recalibrating X/Y/Z targets	18
14. Recipe C — Adding waypoints to the road graph safely	19
15. Recipe D — Testing UI changes without crashing the render loop	21
16. Save, export and release	22
17. Console API quick reference	22
Part III — Future AI Agent Rules of Engagement	24
18. Prime directives	24
19. Memory-safety protocol	24
20. Performance danger zones	25
21. Scene-graph and runtime invariants	26
22. Change protocol for AI agents	27
Part IV — PWA Offline Shell & Campus Analytics	28
23. What the build changes	28
24. The offline model	28
25. CampusAnalytics	29
26. Verification	29
27. Rules for agents working on the PWA layer	30
Appendix A — URL flags for testing	30
Appendix B — Glossary	31

Part I — Current Status & System Architecture

What the file is, how it boots, how it draws a frame, where its data lives, and how the routing, GPS and governor subsystems work today.

1. Executive summary

SastraMap is a single, self-contained HTML document that renders an interactive 3D model of the SASTRA Deemed University campus in Thanjavur (10.73° N) in any WebGL browser. Everything is in one file: the three.js r128 engine, about 76,000 lines of hand-written JavaScript, all CSS, every procedural texture painter and the saved campus layout. There are no image, model or data files. Every surface is painted into a canvas at boot. The Content-Security-Policy allows no network calls at all (`connect-src 'none'`). The only external origins are Google Fonts.

The code is heavily annotated. Nearly every subsystem starts with a banner comment (`/* ===== TITLE ===== */`) that records the design intent, measured numbers and the history of past bugs (HOTFIX, ROOTFIX, MATHFIX, AAA markers). Treat those banners as the specification. This document cites them by title and line number.

Metric	Current value (this revision)
File	4,500,857 bytes · 78,757 lines · 24 inline <code><script></code> blocks · 2 static <code><style></code> blocks (plus styles injected at runtime by each module)
Engine	three.js r128, inlined and minified on line 1958 (603 KB on one line)
Hand-written JS	Host engine 2165–29085 (26,921 lines); World Editor 29113–63000 (33,888 lines); 18 add-on modules 63080–78755
Scene size (measured in code)	~4,945 Object3Ds, of which 4,757 never move; ~4,680 draw calls at the spawn view before the static batcher
Procedural textures	~470 canvases (~275 MB of pixels). They are cached in IndexedDB after the first visit, which saves 40–45% of a desktop boot
Survey data (code)	34 BUILDINGS · 5 GROUNDS · 15 ROADS · 13-vertex BOUNDARY, in satellite pixel space
Saved layout (JSON)	v7 · 41 host transforms · 290 World-Editor placements · 551 retired props · buildStamp <code>Bmuds6ios-9kbp</code>
Card dictionary	69 SASTRA_BUILDINGS records, 14 of them academic records generated from campusData; 60 BUILDING_LABELS (13 not yet bound to an object)
Road graph	155 waypoints · 171 edges · 14 calibrated building targets
Shader programs	74 on desktop, 70 on phones. The code keeps this count flat on purpose (see §5.4)
Build type	DEV build: the payload has no <code>data-viewer="1"</code> flag, so the World Editor mounts on desktops wider than 900 px and saves to localStorage

1.1 Status at a glance

Area	State	Notes
Rendering, governor, boot	Production-hardened	Many measured fixes are in place: thermal ladder, render-on-demand rest tiers, shadow freeze, parallel shader link, texture cache, GPU hand-back on unload.
Road route + live GPS origin	Complete	Dated 2026-09-26/27 (AAA5, AAA6). The graph is static and only 14 buildings are calibrated.
Grid navmesh router	Complete	Sits behind the Route button. It is independent of the card route.
Card data	Placeholder values	labs, classrooms and sponsor are null in all 69 records (Nirman Vihar carries a room list, JVC hand-written tabs). SASTRA_CIVIL is filled only for Nirman Vihar (8,475 m ²) and JVC (10,806 m ²). EU_RESTROOMS is a placeholder.

Area	State	Notes
GPS anchor	Approximate	Taken from public maps, not a survey. Correct it on site with the "Fix" tap or with <code>SastraGeo.calibrate()</code> .
Hidden legacy UI	Intentionally hidden	The <code>#uiPurge</code> stylesheet hides the FREE camera, the search box, the category chips and the minimap. Their elements stay in the DOM because live code still reads them.

2. File anatomy

The file is a chain of classic scripts that share one global scope. The host engine declares top-level `let/const` bindings (`scene`, `camera`, `renderer`, `camT`, `RG`, `BUILDINGS` and so on). Later blocks reach them defensively with `typeof x !== 'undefined'`. Every block after the host is an IIFE that talks to the rest of the page through `window.*` APIs and DOM events.

Lines	Block (banner title)	Responsibility / public surface
1–20	Head metadata	CSP, X-Frame-Options, nosniff, viewport, title
21–711	SASTRA TEXTURE CACHE	IndexedDB persistence of painted canvases; <code>window.__campusTex</code>
712–789	Fonts + RESILIENCE + ANTI-SNOOPING	Frame-buster, F12 / Ctrl+Shift+I/J / context-menu block, WebGL context-loss guard (patches <code>getContext</code>)
790–1846	Static CSS	Tokens, HUD, legacy skin, World-Editor shell CSS, coarse-device glass budget, <code>#uiPurge</code>
1848–1952	Body markup	<code>#app</code> (canvas host), <code>#boot</code> overlay, HUD buttons, retired legacy hosts
1953–1959	three.js r128	Inlined library. Never edit.
1960–2164	LAUNCH · PRECISION LOADER	Boot progress bar; <code>window.__lxLoad</code>
2165–29085	HOST ENGINE	Seed, survey data, engine, governors, async boot, world build, navmesh A*, post-FX, camera, picking, render loop, God-Mode layer, bespoke landmark installers (Library, Main Entrance, Chanakya, East Complex, VKJ, Vidya Vihar, ASK/SHRI, Transit Hub, Nirman Vihar, JVC), traffic engine, Character and Vehicle modes
29113–63000	SASTRA WORLD EDITOR v2.0	Asset modules (flora, roads, props, lights, gizmo), WEC core, shell, persistence, export, static batcher, shader pre-warm; <code>window.WorldEditor</code>
63080–65771	GLOBAL BUILDING LABELS	3D sprite signs, glass name tags, click-a-name, drag-to-place
65801–66010	OUTER HOSTEL LAND	Ground block under the outer hostel sector
66049–68291	SASTRA BUILDING CARD	<code>campusData</code> , <code>SASTRA_BUILDINGS</code> , <code>SASTRA_CIVIL</code> , glass panel; <code>window.SastraCard</code>
68325–68702	SMART CAMPUS	Fast travel and category highlight (<code>#scDock</code>)
68703–68962	ARCH HUD	North compass and CAD site grid
68963–69209	UX POLISH	Momentum, hover cue, double-tap home, canvas fade-in
69213–69580	LAUNCH · GESTURE LOCK · TOOLTIP	Touch gesture lock, hover tooltip, day/night toggle
69624–70518	VISITOR EXPERIENCE	Shareable links, landmark pins, Security View
70550–71215	ACCESS & CAMPUS LIFE	Campus search (<code>#axSearch</code>), accessible-route view
71253–72429	EVERYDAY UTILITY	GPS blue dot, food/restroom tags, Zen; <code>window.SastraGeo</code>
72430–73071	AAA · SELECTION GLOW	Neon emissive highlight of the open card's building
73072–74477	AAA5 · ROAD ROUTE	Road-graph A*, fillets, Catmull-Rom tubes; <code>window.SastraRoute</code>
74478–75000	AAA · WEATHER	Dust motes / rain; <code>window.SastraWeather</code>
75001–75367	AAA · DEPTH OF FIELD	CSS backdrop lens while the side panel is open
75368–76259	AAA · RADAR	Round glass minimap

Lines	Block (banner title)	Responsibility / public surface
76260–76400	FRUSTUM-CULL ENFORCER	<code>window.__mapCull</code>
76401–77971	AAA6 · LIVE ORIGIN & PIN LAYER	From pill, POI pins; <code>window.SastraPOI</code>
77972–78755	GLASS SETTINGS	Gear + frosted panel, Eco Mode; <code>window.SastraSettings</code>
78757	#sathizz-map-data	URL-encoded JSON layout payload (87 KB, one line)

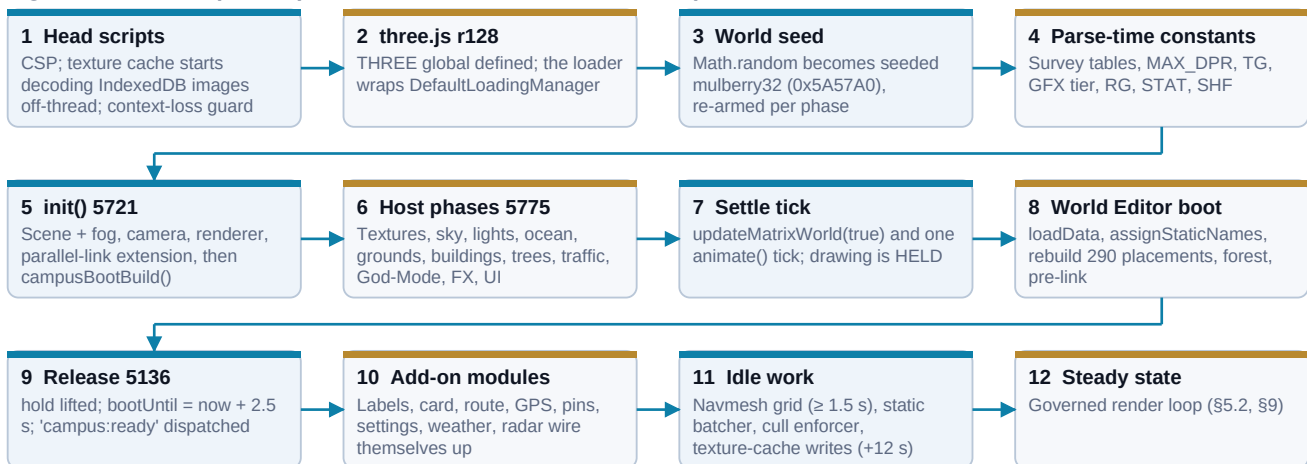
Line numbers drift

All line numbers in this document refer to this exact revision (78,757 lines, buildStamp Bmuds6ios-9kbp). After any edit, find a section by searching for its banner title (for example `RENDER GOVERNOR` or `AAA5 · ROAD ROUTE`) or its function name. Do not rely on the numbers.

3. Boot sequence

The boot is asynchronous and happens in phases, so the overlay can paint straight away while the world streams in behind it. Drawing is held until the World Editor has rebuilt the saved layout and the final light rig is linked. After that, `'campus:ready'` fires and every add-on module finishes wiring itself.

Figure 1 — boot sequence (line numbers are function definitions)



Two things about the boot are easy to miss:

- **Determinism.** The first statement of the host script replaces `Math.random` with a seeded generator. `__campusSeed(tag)` re-seeds it for each construction phase (FNV-1a of the tag). The same campus is built on every load: 481 procedural call sites depend on it, and so do the forest collision system and the texture cache. Runtime modules build their objects under the native generator (`Math.__nativeRandom`, through a `withNativeRandom()` helper).
- **Program linking.** Shader links used to take 18.2 s of a 35 s cold load. The file now links in parallel (`KHR_parallel_shader_compile`, `pcPrelinkScene` / `pcWhenLinked`) against the final light rig. EARLY LINK forecasts the lamp posts from the saved layout, so there is no second relink.

4. Coordinate system & the JSON Data Dictionary

4.1 Coordinate system

Axes	+X = east, -Z = north (north is "up" on the satellite image), +Y = up. World (0,0,0) never moves.
Survey space	Host geometry is authored in the pixel space of a 2000 × 1308 satellite capture. <code>S = 0.2</code> , <code>CX = 1000</code> , <code>CY = 654</code> (line 2274).
Pixel → world	<code>world.x = (px - 1000) * 0.2</code> <code>world.z = (py - 654) * 0.2</code>

World → pixel	$px = x / 0.2 + 1000$ $py = z / 0.2 + 654$
Scale	Host header: 1 world unit $\approx$ 2 m (1 px $\approx$ 0.4 m, $M_PER_PX = 0.4$). The GPS fit and the card's walk distance use 1.13 m per unit (M_PER_U). See the known-issues list.
Heights	$FL00R_H = 2.35$ u per storey; host walls start at a 0.85 u plinth. Road tops sit at y 0.02–0.32.

4.2 Layer 1 — Host survey tables (code, lines 2373–2700)

These are hand-traced constants. They drive the host-built buildings, the tree exclusion zones, the navmesh stamp, the search index and the legacy minimap.

Constant	Shape	Notes
BOUNDARY (2373)	13 [px, py] vertices	Lofts the ground, the soil plinth and the boundary wall. The vertex count must stay 13 : the wall emits two meshes per segment, and a new vertex rennumbers every later ENV_ITEM.
GATE, PARKING	point / rect	Main entrance on the Thanjavur Road frontage; legacy car park.
BUILDINGS (2382)	34 records	{id, name, ta, cat, label, hero?, labelShort?, custom?, desc, descTa, parts:[{poly, hole?, floors, wall, roof, dome?, noClutter?}]}]. wall: cream brick. roof: flat solar terracotta bluemetal whitemetal. Several entries survive only as footprints: an HD installer deletes their meshes and builds the real model.
GROUNDS (2647)	5 records	Football, cricket "Canopy", practice ground, basketball, lawn. detached:true keeps the polygon for search, forest and navmesh. The mesh is a placeable editor asset (weAssetId).
ROADS (2680)	15 polylines	1 highway (w 170 px, the 4-lane carriageway) and 14 internal lanes (w 11–17 px).

Polygon helpers: `rect(x1,y1,x2,y2)`, `rrect(cx,cy,w,h,deg)`, `circle(cx,cy,r,seg)`, `insetPoly(poly,d)` and `pointInPoly`. Polygons are wound as `rect()` winds them.

4.3 Layer 2 — The embedded layout payload (line 78757)

The last line of the file is `<div id="sathizz-map-data" style="display:none">`. It holds the World Editor's layout as `encodeURIComponent(JSON.stringify(data))`. Decoded, it is the "JSON Data Dictionary" that places everything that is not host-built:

Key	Type / count	Meaning
version	7	Layout schema version
buildings	object · 41	Host transforms keyed by object name: 39 GROUP_<hostId> + 2 WE_<uid> . Value: {pos:[x,y,z], rot:[x,y,z] (Euler, rad), scl:[x,y,z]}. Examples: GROUP_main-gate at scale 0.5, GROUP_n-sheds at 0.75.
custom	array · 290	Placements {uid, asset, p:[x,y,z], r: yaw_rad, s:[x,y,z]}. uid format WE_<asset>_<base36 time>_<seq> (also _inplace_n for decoupled host pieces).
instances	array · 0	Brush batches {a, p, r, s} (InstancedMesh). Unused in this build.
deletedProps	array · 551	Host meshes hidden by the editor: 404 ENV_ITEM_<n> and 147 CHILD_<hostId>_<i> .
buildStamp	string	Bmuds6ios-9kbp . Lineage stamp: a localStorage save outranks the payload only when the stamps match.
fx	object	Post-FX snapshot: {v:1, mode:0, exposure:.88, bloom:.18, vig:.32, bright:.94, partOp:.5, warmGlow:.32, labels:false}
forestRestored, recoveredV1, inplaceV1	1	One-shot migration latches. Keep them.

Most-used assets in **custom**: wall-boundary-cream 36, oak 24, custom-road-straight 22, path-concrete 19, path-paver-grey 9, palm 9, path-paver-red 9, planter-round 8, custom-road-right-90 7, custom-building-5 5,

gb_umbrellas 5. The rest are one-to-four-off landmark assets (custom-chith-vihar, custom-knowledge-wings, custom-kc-canteen, custom-velaa-cafe, the southern canopy food court, and so on).

Load precedence (`readEmbedded()` 61249, `loadData()` 61304). The embedded payload is compared with the per-file localStorage key `SASTRA_CAMPUS_DATA_V7:::<filename>`. The key falls back once to the old shared key, then to v6. localStorage wins only when its `buildStamp` equals the payload's. A viewer build (`data-viewer="1"`) never reads localStorage. `save()` (61174) rotates five backups (`::bak1..5`). If the load could not rebuild every placement, it writes to `::quarantine` instead of over the good save. `exportHTML(finalBuild)` (61653) clones the document, strips the runtime DOM, stamps a new buildStamp and embeds the payload. It saves the result as `SastraMap_Dev.html`, or as `SastraMap_Final.html` with `data-viewer="1"`.

4.4 Layer 3 — The card data dictionary

Structure	Where	Shape
<code>campusData</code>	66108 (frozen)	Code → <code>{branches, classes, labs, desc}</code> strings for 14 academic buildings: VV, VKJ, SOC, LTC, SASHE, TDC, TIFAC, CV, SOE, CHANAKYA, POLY, ASK1, ASK2, ASK3. Room format: "Ground: 101-109 First: 201-214". Lists are comma-separated; commas inside parentheses stay with their item.
<code>CAMPUS_CODES</code>	after <code>campusData</code>	Code → record key (VV: 'vidyut_vihar', SOE: 'vidhya_vihar' ...). <code>applyCampusData()</code> builds the Branches / Classrooms / Labs / Details tabs.
<code>SASTRA_BUILDINGS</code>	66190	69 records <code>{key, name, sub, labs, classrooms, sponsor, desc?, rooms?, tabs?, frame?, match:[]}</code> . <code>match</code> namespaces: <code>l</code> : label id, <code>w</code> : placement uid, <code>a</code> : asset id, <code>h</code> : host id. Namespaces matter: ' <code>h:shannugha</code> ' is the Polytechnic, ' <code>l:shannugha</code> ' is the canteen.
<code>SASTRA_CIVIL</code>	66348	<code>{elev, area}</code> per record key. null renders as "--". Live edit: <code>SastraCard.setCivil('library', {elev: 61.2, area: 5400})</code> .
<code>BUILDING_LABELS</code>	63102	60 signs <code>{id, name, sub?, anchor, lift?, off?}</code> (13 with <code>anchor: null</code> : named from photos, object not yet identified; fix with <code>labelBindSelected(id)</code>). The anchor resolves in this order: buildingGroups key, object name (placement uid), ED.objects, userData ids.
<code>ROAD_TARGETS</code>	73305	14 calibrated route ends (§7.3).

4.5 The naming contract (critical)

`assignStaticNames()` (60392) runs once, before the editor adds anything. It names host groups `GROUP_<id>` and their children `CHILD_<id>_<i>`. It then numbers **every mesh in scene**. `traverse()` **order** as `ENV_ITEM_<n>`, skipping only meshes flagged `userData.weIndexExempt`. The shipped layout addresses objects through these names: 404 hidden props and the saved host transforms. Adding, removing or reordering one host mesh that the traversal reaches before a numbered one changes every later index. The layout then quietly hides and moves the wrong objects all over the campus. The file protects this invariant in more than 20 places (the flower batches, the navmesh tube, the labels, the Main Entrance padding its subtree back to exactly 348 meshes, and more).

5. The Three.js / WebGL rendering pipeline

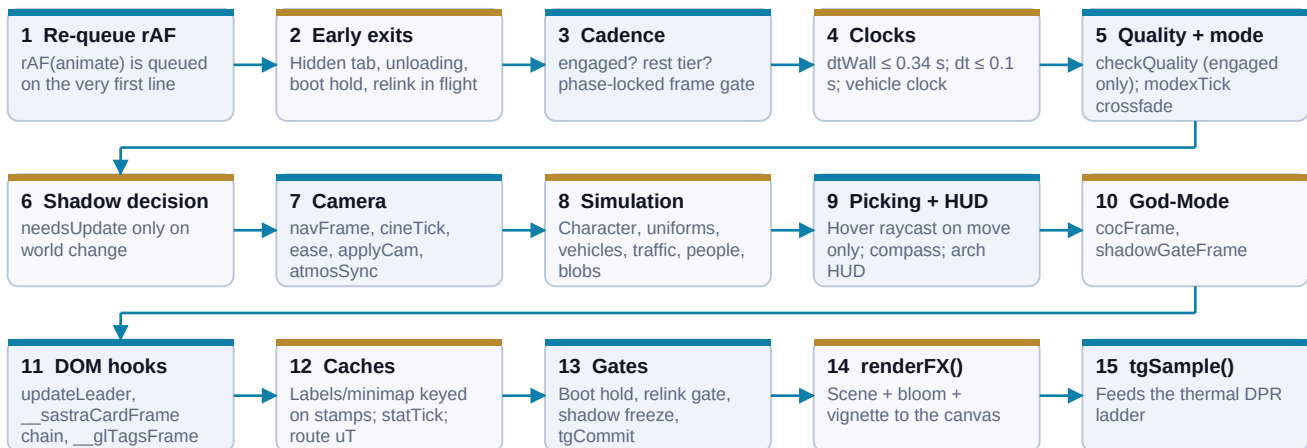
5.1 Renderer configuration

Renderer	<code>campusMakeRenderer({antialias: GFX.antialias, powerPreference: "high-performance", stencil: false})</code> . On the phone tier it reports <code>MAX_TEXTURE_SIZE</code> clamped while three.js is constructed, so every texture and shadow map is capped by three's own resize path.
Pixel ratio	<code>targetDPR() = min(native, MAX_DPR = 2, GFX.dprCap, current TG rung)</code>
Colour	sRGB output, ACESFilmic tone mapping, exposure 0.94 (the payload fx sets 0.88). <code>linearizeMaterials()</code> converts sRGB hex colours to linear once, at boot.
Shadows	PCFSoftShadowMap. <code>shadowMap.autoUpdate = false</code> : the loop decides every shadow pass. The sun map starts at 2560 ² in <code>buildLights().hdBoost()</code> (PART C, about line 18081) raises it to 4096 ² on desktop and 2048 ² on handhelds; the low tier uses 1024 or 512. <code>fitSunShadow()</code> fits its frustum to the campus.

Camera	PerspectiveCamera fov 36, near 1, far 6000. Custom orbit rig: cam eases toward camT (tx , ty , tz , dist , az , el). There are no OrbitControls. The spawn distance (382) is the zoom ceiling.
Lights	Hemisphere (0.55), sun directional (2.05, casts shadows), fill directional (0.42). Lamp posts come from WE_LIGHTS within a fixed, program-stable budget.
Sky / fog	Gradient ShaderMaterial sphere (r 2600), 2,600-star Points. THREE.Fog is created in init() before any link, because fog is part of every lit program's key.

5.2 Anatomy of one frame — animate() (line 11849)

Figure 2 — the order of work inside one drawn frame



Each step is ordered on purpose. The banner comments inside `animate()` explain why. Examples: the leader line runs before the chip writes, to avoid a forced reflow. `statTick()` runs last before the draw, so a thaw is never a frame late. `tgCommit()` sits right before `renderFX()`, because `setPixelRatio` clears the canvas and only a draw in the same frame repaints it.

5.3 Post-processing chain (renderFX, line 9906)

Pass	Target	Detail
Scene	FX.rtScene (full res)	sRGB, depth. MSAA 0/2/4 samples from fxMsaa() : 0 at native ratio, 2 at the 1.5 rung, 4 below. Desktop WebGL2 only; never in Eco.
Bright pass	FX.rtQA (¼ res)	Luma smoothstep from a 0.82 threshold
Blur H / V	rtQB → rtQA	Separable 9-tap Gaussian. Tap step $2.2 * \text{ratio} / 1.25$ keeps the glow the same size in CSS pixels
Composite	Canvas	scene + bloom * uBloom , then a radial vignette uVig
Fallback	Canvas	qStep ≥ 3 disposes all three targets and renders the scene straight to the back buffer

There is no EffectComposer. The inlined r128 bundle does not include one, and every extra target or program costs thermal budget. New effects reuse this chain: the selection glow feeds emissive light into the same bloom.

5.4 The program-stability model

In `three.js` every distinct material "program key" is a separate GPU shader link. A cold link blocks for 100 ms or more. The file keeps the program count flat (74 desktop / 70 phone) and avoids links during a frame:

- **Parallel link at boot** (PARALLEL SHADER LINK 4815, BOOT HAND-OVER 5021, EARLY LINK 5163).
- **Runtime relink** (5043): light counts are part of the program key, so placing or erasing a lamp post relinks every lit program. `__campusRelink()` links in the background. The loop keeps simulating, stops drawing, then resumes.
- Anything that changes a key at runtime causes a visible hitch: a new material class, map/no-map, vertexColors, the fog flag, toneMapped, side, onBeforeCompile, or a light count. Add-on modules therefore reuse keys that already exist. The route draws only with MeshBasic (BackSide) and LineBasic with toneMapped:false. The pins

use the same `SpriteMaterial` key as the lamp glows.

5.5 CPU and draw-call savers

Mechanism	Where	What it does
Change stamps	CHANGE STAMPS 3569	<code>WORLD_STAMP</code> , <code>WARM_STAMP</code> , <code>CAM_STAMP</code> . Every view-independent cache keys on them, so a still frame recomputes nothing.
Static matrix freeze	STAT 3728–4217	Probes 90 frames (2.6 s), then sets <code>matrixAutoUpdate = false</code> on everything that never moved. A rolling audit covers $\frac{1}{4}$ per frame. <code>bumpShadows()</code> calls <code>statReset()</code> and re-learns. A subtree short-cut skips frozen subtrees entirely.
Static batcher	World Editor 61828	Merges eligible placed meshes per material in 48-u cells into <code>WE_BATCHES</code> . The originals are parked on layer 13 : never drawn, still pickable. The selected placement is unbatched straight away. <code>__campusBatch.off().on()</code> .
Culling	FRUSTUM-CULLING AUDIT 3959; ENFORCER 76260	Real bounding spheres and <code>frustumCulled = true</code> on every eligible mesh; <code>__mapCull()</code> .
Gates	4616, 4655, 4704	Landmark detail gate, ghost gate, local shadow gate (the largest single GPU saving).

5.6 Scene flags every module respects

Flag	Effect
<code>userData.weIndexExempt</code>	Not named or counted by the <code>ENV_ITEM</code> numbering (§4.5)
<code>userData.weNoFreeze</code>	Never frozen by STAT; keeps its own shadow cadence (animated or driven objects)
<code>userData.weNoCull</code>	Left alone by the frustum-cull enforcer
<code>userData.aaaFx</code>	An effect object: pickers, box measurement and footprint rays skip it
layer 13	Batched originals. No camera renders it; editor and card raycasters opt in with <code>layers.enable(13)</code>
<code>o.raycast = function () {}</code>	Decorative objects (labels, route, pins) can never be picked

6. The glassmorphism UI layer

All user-facing UI is DOM composited over the WebGL canvas. The only text drawn in WebGL is the building-name sprites, and they now serve only as anchors for DOM "glass name tags". The glass recipe is used consistently (62 `backdrop-filter` declarations): a translucent dark gradient (`rgba(16,22,36,.58-.88)`), a 1 px white border at 12–25% alpha, `backdrop-filter: blur(10–20px) saturate(130–170%)`, a large soft shadow, gold accents (`#E6C35C` / `#f0d38e`) and a pure cyan `#00FFFF` for routes.

Component	Owner block	Notes
<code>#sInfoCard</code> building card	SASTRA BUILDING CARD	Desktop: right side panel (<code>.panel</code>). Phone: bottom sheet (<code>.sheet</code>). z 33, blur 20 px, saturate 170%. <code>#sInfoDot</code> / <code>#sInfoStem</code> stay pinned to the measured box centre of the building.
<code>#glTags</code> <code>.gl-box</code> name tags	GLOBAL BUILDING LABELS	blur 16 px, z 12. Fade in 0.5 s; fade out 3.5 s (or 0.5 s with <code>.gl-quick</code>). Opacity lives on the box, never on an ancestor.
<code>#axSearch</code> + <code>#pxBar</code> pills	ACCESS; AAA6	Search and the From pill (route origin); Food / Restrooms / ATM / Bus / Parking pin filters; blur 14 px.
<code>#euDock</code> , <code>#euChip</code> , <code>#euMe</code>	EVERYDAY UTILITY	Locate, Food, WC and Zen buttons; location chip; the blue dot.
<code>#gxGear</code> / <code>#gxPanel</code>	GLASS SETTINGS	One gear (z 58) opens a frosted panel. Each row proxies an original button through <code>.click()</code> and mirrors it with a <code>MutationObserver</code> . Eco, POI and Compass toggles.
<code>#dof</code> lens, <code>#radar</code>	AAA W7, W8	The CSS depth-of-field lens (z 11, blur 6 px, radial mask). The round glass minimap (168 px).

z-index stack: canvas (#app) < scrims 6 < #dof 11 < #glTags 12 < POI pins 24 < radar / #euDock 26 < #euChip 27 < card dot 32 < #sInfoCard 33 (#axSearch 34 while its menu is open) < #gxGear 58 < #lxlne 400.

Performance rules built into the UI: backdrop-filter is switched off at max-width 900 px (the coarse-device glass budget, lines 1790–1810, and the card's own media queries). DOM transforms are written only when a value moves by ½ px. Viewport sizes are cached on resize, never read inside the frame. `html.rg-rest` pauses CSS animations while the render loop rests, so the compositor stops re-blurring. `@supports` fallbacks and `prefers-reduced-motion` are respected everywhere.

Integration surface: events `campus:ready`, `campus:loaded`, `campus:revealed`, `sastra:card {open, key, part}`, `sastra:origin`, `sastra:route`, `sastra:weather`; frame hooks `window.__sastraCardFrame` (a chain), `__glTagsFrame`, `__archHudFrame`; loop wake `window.__mapWake()`. Every module first removes stale copies of its own DOM ids, because exports bake the DOM. Its ids and body classes are also listed in the exporter's strip list (`exportHTML`, about line 61706).

7. Pathfinding: A* routing and Catmull-Rom splines

The file contains **two independent routers**. Keep them apart when you make changes:

	Grid NavMesh A* (NMESH)	Road-Graph A* (RouteCore / SastraRoute)
Code	Host, 7931–9460	AAA5 block, 73072–74477
Trigger	The "Route" button (#bRoute), then pick two buildings	Opening any building card (automatic). The start is the live GPS fix, a declared spot or the Main Gate
Search space	Raster of 1.5 u cells, ~166,400 cells, typed arrays	155 measured waypoints, 171 edges
Edge cost	Terrain cost field: road 0.35, pave 0.80, grass/turf 8.0	Length × class: asphalt 1, concrete 1.1, grey pavers 1.35, red pavers 1.6, connector 1.25
Heuristic	Octile × C × 0.35. Consistent, so the path is optimal for the cost field	Euclidean. Admissible because every multiplier is ≥ 1
Queue	Indexed binary min-heap, generation stamps, zero GC	Binary heap of [f, id] with lazy deletion
Smoothing	Cost-priced string-pull, then centripetal Catmull-Rom with subdivide-and-repair (≤ 24 passes)	Corner fillets (3 points per turn) and densify every 10 u, then centripetal Catmull-Rom
Drawn as	One TubeGeometry (r 0.42) with a shared chevron ShaderMaterial	A bundle: core tube r 0.30, halo r 1.05, flow tube r 0.50, a 1 px line, two rings, a beacon, a comet head
Distance	<code>length × 2.0 m</code>	<code>length × 1.13 m</code> ; ETA at 80 m/min
API	<code>campusRoute</code> , <code>campusRouteBuildings</code> , <code>NMESH</code> , <code>__navStats</code> , <code>__navRebuild</code>	<code>SastraRoute</code> .{ <code>on</code> , <code>set</code> , <code>plan</code> , <code>origin</code> , <code>reroute</code> , <code>snap</code> , <code>samples</code> , <code>state</code> }

7.1 Grid NavMesh A* in detail

- **Build** (`nmBuild` 8472). Scheduled at least 1.5 s after boot, on idle, and only once frames are on screen. It is never on the boot path. Rule 1 scan-fills the boundary. Rule 2 stamps road centrelines as cost (never as walls). Rule 3 stamps each of the ~1,031 `FOREST_SOLIDS` boxes by class: planting-exclusion zones are skipped; grounds and parking get `C_PAVE`; turf gets `C_TURF`; building polygons are blocked with a 1 px NAV pad; AABB blockers are size-gated at 30 u. Only boxes inside the pedestrian slab (`FOOT` 0.55 to `HEAD` 2.10) block. Tree trunks are inflated by 0.55 u. Connected components are labelled and endpoints must lie in the main one.
- **Endpoints**. `nmSnapToward()` marches from the building centroid toward the other endpoint in half-cell steps and takes the first walkable cell. That cell is the door that faces the journey. The fallback is a bounded spiral (`nmNearest`, ≤ 140 rings).
- **Search** (`nmAStar` 8710). 8-connected with no corner cutting. Step cost = $\text{step} \times \frac{1}{2}(k_{\text{from}} + k_{\text{to}})$. `MAX_EXPAND = 140000`. When that is exceeded, the search returns the best node by heuristic and flags the route *partial*. Every search first clears exactly the heap slots the previous search touched. The generation stamp re-baselines before it can wrap at 2^{31} .
- **String-pull** (`nmLOS` 8929). A supercover-Bresenham line is priced in the same units as the A*. It is rejected outright if it crosses a cell dearer than `LOS_MAX_K = 0.80 + 1e-4` (the epsilon absorbs float32 error). It is accepted only if it

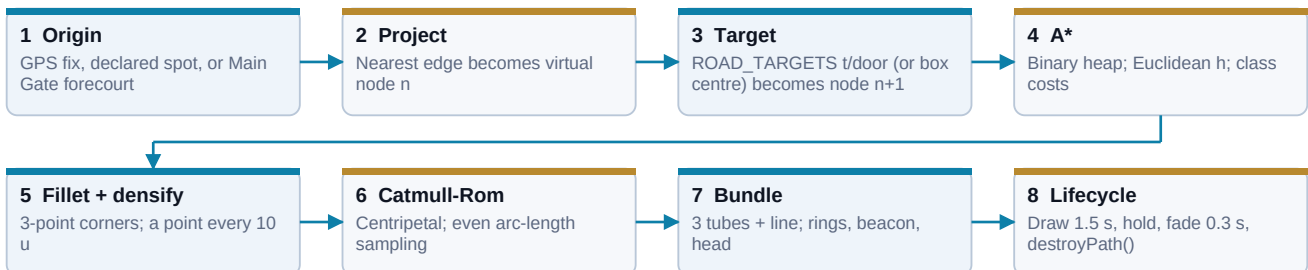
costs no more than `PULL_TOL = 1.15 ×` the chain it replaces. This stops the smoother cutting a road bend across a playing field.

- **Spline and repair** (`nmDrawRoute 9055`). `new THREE.CatmullRomCurve3(pts, false, 'centripetal', 0.5)`. The curve is sampled every half cell. The first sample that falls on a non-walkable cell names the span `floor(t × (N-1))`, and the raw A* cell at that span's midpoint is spliced back in. This repeats up to 24 times. The tube is `TubeGeometry(curve, clamp(1.15·L, 24, 600), 0.42, 8)` at y 0.85.
- **Lifecycle**. One mesh named `NAVMESH_ROUTE_TUBE` (`weIndexExempt`). On a new route the old geometry is **disposed** and the new one attached to the same mesh. The material is built once and shared. Per frame the router writes a single uniform (`NMESH.mat.uniforms.uT`).

7.2 Road-Graph A* in detail (AAA5 / AAA6)

- **ROAD_NET** (73255). `nodes: [x, y, z]`, where y is the road-top height there. `edges: [a, b, cls]`. Classes 1 asphalt (105 edges), 2 concrete (38), 3 grey pavers (0), 4 red pavers (11), 5 connectors (17). The graph was measured offline from the placements' own triangles and verified: all 14 academic routes stay on paving when sampled every 0.5 u.
- **ROAD_TARGETS** (73305). `key → {code, t:[x,y,z], door:[x,z] | null, ref:[x,z]}`. `t` is the volume/plan centroid of the building's meshes. `door` is the network point on its frontage. `ref` is the box centre the calibration was measured against. If the building later moves in the editor by more than 1 u (`MOVED`), `t` and `door` shift by the same delta. **ROAD_GATE** (73321) holds the gate forecourt start and its threshold.
- **Plan** (`plan()` 73727). The origin and the door (or target) are each *projected* onto the nearest edge. They become virtual nodes `n` and `n+1`, joined to both ends of their edge with proportional costs. A* runs between them. For an uncalibrated card, the route stops at the road when the nearest road is more than 20 u away (`TAIL_MAX`), and no route is drawn when it is more than 60 u away (`TAIL_NONE`). An origin more than 60 u off the network (`SPUR_MAX`) starts at the nearest kerb.
- **Smoothing** (`RouteCore.smooth`). Points closer than 0.4 u are dropped. Turns under 0.12 rad (~7°) stay plain waypoints. Every other corner becomes three points: in at `r`, the midpoint of the quadratic Bézier, out at `r`. `r` is the class's fillet radius (asphalt 6, concrete 3.5, others 3), scaled by the turn and capped at 45% of either leg. Straights get a point every 10 u. The curve is `CatmullRomCurve3(pts, false, 'centripetal')` with `arcLengthDivisions = clamp(2·len, 200, 2400)`, lifted 0.6 u above the road.
- **Draw**. The draw-on takes 1.5 s (easeOutCubic through `setDrawRange`), then the tube breathes and dashes flow along it. Closing or switching the card fades it over 0.3 s, then destroys it. Tubular segments: `clamp(1.2·len, 64, 900)` (`clamp(0.6·len, 40, 420)` on phones and in Eco).
- **Moving origin** (AAA6:O6). A GPS dot re-plans at most every 650 ms, and only after moving 2.5 u. `retube()` (74085) disposes the four old geometries and attaches four new ones to the same meshes. A jump of more than 40 u redraws the route from the start.

Figure 3 — road-route pipeline (AAA5 / AAA6)



7.3 Route memory model

Each route is a **new** group `AAA_routePath` with four meshes, four geometries and four materials (`newBundle 73935`). `destroyPath()` (73971) runs in a fixed order: (1) remove the group from the scene first, so no compile pass can meet a disposed material; (2) dispose all four geometries; (3) dispose each material only if `matSafe(m)` shows that every program it holds has another user. Otherwise the material goes into **GRAVE** and is freed on a later frame. The **KEEPERS** are a hidden mesh and line on "PARK" materials, drawn once. They keep the MeshBasic and LineBasic programs alive, so a dispose never drops a shared program to zero users and the next route links nothing. A

five-node SLOT stands in when no path exists, so the scene's node count never changes (§19, rule M6). The counters in `SastraRoute.state().mem` let you verify that nothing leaks.

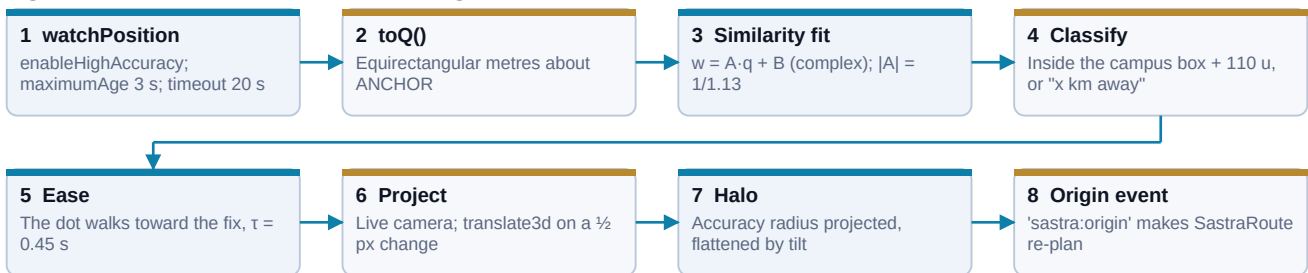
Catmull-Rom rules that apply to both routers

Use **centripetal** parameterisation ($\alpha = 0.5$). It avoids cusps and self-intersections on unevenly spaced points. Never feed raw lattice staircases or bare right-angle corners into a spline: string-pull or fillet first, or it will overshoot off the road and through trees. three's `getPoint(t)` is uniform across segments. Use `getSpacedPoints / arcLengthDivisions` when geometry needs even spacing.

8. Live GPS tracking (the Blue Dot)

Owned by EVERYDAY UTILITY (71253–72429), API `window.SastraGeo`. The dot is an HTML element (`#euMe`) positioned over the canvas. It is not a 3D object, so it costs no draw call, no program and nothing in the scene graph.

Figure 4 — GPS fix to blue dot to route origin



- **Start.** Tap `#euLocate`. This needs a secure context (https or localhost). After `campus:ready` + 900 ms the watch starts quietly if the Permissions API already reports "granted" (`?geo=off` prevents that).
- **Projection** (`toQ`, `refit` 71676, `toWorld`). East metres = $\Delta\text{lon} \times 111,320 \times \cos(\text{lat}\theta)$ and south metres = $-\Delta\text{lat} \times 110,574$ about `ANCHOR = {lat: 10.7296, lon: 79.0194}` (71655, approximate). As complex numbers, $w = A \cdot q + B$: $|A| = 1/1.13$ units per metre, and $\arg A$ is the compass's `trueNorthDeg`. B is the measured centre of 'main_entrance' (fallback $(-105, -77)$).
- **Calibration.** "Fix" means the user taps where they really stand. `addCal()` keeps at most two points in `localStorage["sastra.geo.cal.v1"]`. One point re-anchors B . Two points at least 60 m apart solve $A = \Delta w / \Delta q$, which fits rotation and scale. The fit is accepted only if the scale stays within 0.75–1.33, so GPS noise cannot pass for a 25% scale error.
- **Rendering** (`meFrame` 72250, chained on `__sastraCardFrame`). The dot eases toward the fix (exact under reduced motion) and calls `__mapWake` until it lands. It is projected with the same camera matrices as the frame. The accuracy halo is `acc / 1.13 u`, projected along the camera's level right-hand axis and flattened by the view tilt. A fix older than 30 s marks the dot stale.
- **Follow mode** (double-tap). `followFrame()` writes `camT.tx/tz` to the dot. Any other camera flight (card, search, fast travel) releases it.
- **Origin arbitration** (AAA6:08). `SastraGeo.origin()` returns the live fix while GPS is on and on campus. Otherwise it returns a declared place ("Set my location": a named place or a tap on the map, kept in `sessionStorage["sastra.geo.origin.v1"]`). With neither, it returns null and the route starts at the Main Gate. Every change fires 'sastra:origin'.
- **Testing away from campus:** `SastraGeo.simulate(10.7296, 79.0194, 6), SastraGeo.state(), SastraGeo.toWorld(lat, lon), SastraGeo.resetCalibration(), ?geo=lat,lon` to move the anchor.

9. Environmental systems & the thermal governor

9.1 Environment

System	Where	Behaviour
Modes	MODE SYSTEM 13165	Golden (0), Day (1), Night (2). <code>applyMode()</code> is the single authority. <code>setMode()</code> is instant (boot, restore, export). The button and M key use a 3 s snapshot-diff-ease crossfade (<code>modexTick</code>). Night windows are injected through <code>onBeforeCompile</code> and add no lights.
Daylight clock	DAYCLOCK 13272	Tints the hemisphere light and scales its intensity by about $\pm 8\%$, from keyframes on the viewer's local hour (smoothstep). <code>?clock=0</code> disables it.
Sun clock	SUNCLOCK 13347	Aims the sun by the solar position at 10.73° N for the local time. The elevation is compressed to $14\text{--}62^\circ$ (Golden $\leq 27^\circ$) so shadows stay readable. <code>?hour=9.5</code> , <code>__campusSun.setHour(h)</code> , <code>?sun=0</code> . Each step raises a shadow refresh.
Atmosphere	ATMOS 6077	Fog colour = $0.87 \text{ horizon} + 0.09 \text{ bottom} + 0.04 \text{ zenith}$ of the sky shader. <code>near = 330 + 0.45 \cdot \text{dist}</code> , <code>far = 1900 + 0.9 \cdot \text{dist}</code> . <code>?fog=0</code> parks the ramp; it never removes the fog (program key).
Weather	AAA W6 74478	One <code>THREE.Points</code> box that rides with the camera: off, motes or rain. Motes by default on desktop; off on phones and under reduced motion. <code>localStorage["sastra.gx.weather"]</code> .

9.2 The thermal governor, layer by layer

Layer	Where	Rule
GFX tier	3478	Decided once, at parse time. The low tier is touch-first, iOS, a mobile UA, or <code>?gfx=low</code> : DPR cap 1.25, no MSAA, textures 512 px (iOS or ≤ 4 GB) or 1024 px, sun shadow 512/1024, no lamp shadows, canvases downsized after painting.
DPR ceiling	<code>MAX_DPR</code> 3314	Hard cap of 2 on every tier. A 3x or 4x phone never allocates 9–16x buffers.
Adaptive retina	<code>TG</code> 3354, sampler 11683	Rungs [2, 1.5, 1.25], collapsed against the native ratio. Windows of 2 s with ≥ 60 engaged samples; a miss is an interval $> 1.45\times$ the gap. DOWN after 2 bad windows ($> 20\%$ misses, median CPU $< \frac{1}{2}$ the gap, ≥ 8 s since the last change). UP after ≥ 30 s clean ($< 2\%$) and ≥ 60 s since a change, at most 2 climbs. A drop within 90 s of a climb pins a floor. <code>?dpr=n</code> pins the ratio; <code>?tg=0</code> disables the ladder.
Cadence ladder	<code>RG</code> 3588	Engaged 60 fps (<code>?fps</code>). Idle 30/24; deep 20/12 after 8 s/4 s; sleep 10/6 after 20 s/12 s (desktop/phone). REST draws zero frames after 30 s/15 s, or after 2.5 s with a card open. The gate is phase-locked, so 60 fps lands on every second vsync at 120 Hz. World time is preserved: dt is clamped at 0.34 s and integrated in steps of at most 0.1 s.
Quality ladder	<code>checkQuality</code> 11650	Warm-up 6 s after boot; windows of 2.5 s; below 30 fps it steps up; step 3 drops the post chain; 2 healthy windows climb back (max 3). It locks once the scene is healthy at full quality.
Shadow refresh / freeze	3681; SHF 4218	The sun map is never redrawn for a camera move. After the probe it is frozen and redrawn only on real change (bumpShadows, mode, sun step, relink, caster signature). Movers leave the depth pass and use one instanced contact-blob layer. <code>?shadows=live</code> restores the old cadence.
Eco Mode	GLASS SETTINGS	Engaged 30 / idle 20 / deep 12 fps, DPR 1, quality ladder paused. <code>localStorage["sastra.gx.eco"]</code> .
Unload	<code>unloadFreeGpu</code> 4555	On beforeunload or pagehide, frees every geometry, material, texture, target and shadow map, so a reload never holds two campuses in GPU memory.

Diagnostics: `__mapFPS(true)` (note: an FPS read-out keeps the loop from resting), `__mapThermal()`, `__mapThermalDpr(i | "auto")`, `__mapRest(ms)`, `__mapCardRest(ms)`, `__mapShadows('live' | 'frozen')`.

10. Observations & known issues

- **Two distance scales.** The host survey and grid router use 2.0 m per unit. The GPS fit, the road route and the card's walking distance use 1.13 m per unit. The same path gives different metre figures in the two routers. Settle this against a site survey before you publish distances.
- **Placeholder content.** labs, classrooms and sponsor are null in all 69 records. SASTRA_CIVIL is mostly null. The host header flags the `desc/descTa` blurbs as placeholders (" $>>>$ EDIT ME"). 13 of the 60 labels have `anchor: null`. EU_RESTROOMS names buildings, not restrooms. ANCHOR is approximate.

- **Static road graph.** Roads moved or added in the World Editor are not routed until ROAD_NET is re-measured. The hostels north of the highway have no road in the graph, so their cards draw no route (TAIL_NONE).
- **Calibration coverage.** Only 14 academic buildings have ROAD_TARGETS. Every other card ends at its box centre, with a capped straight final leg.
- **Dev build.** On a desktop, the editor mounts and auto-saves to localStorage on beforeunload. Visitors should get an "Export Final build" (viewer) file.
- **Developer friction.** The resilience block intercepts F12, Ctrl+Shift+I/J and right-click (as a deterrent only). The CSP forbids fetch, XHR and WebSocket, so any networked feature needs a deliberate CSP change.
- **Legacy retained.** `#card`, `#labels`, `#search`, `#filters` and `#minimap` are hidden or empty but still read by live code. Do not delete them.
- **Unused class.** Road class 3 (grey pavers, cost 1.35) is defined, but no edge uses it.

Part II — Human Developer Onboarding Guide

Step-by-step procedures for the changes people ask for most often, each with a verification step. Read Part I, sections 4 and 5, first.

11. Setting up and daily workflow

1. **Keep one canonical copy under version control**, and give every build a distinct filename. The editor's localStorage key includes the filename (`SASTRA_CAMPUS_DATA_V7:<file>`), so a renamed file starts a new save lineage.
2. **Serve it over localhost, not file://**. Run `python3 -m http.server 8080` in the folder and open `http://localhost:8080/SastraMap_Done.html`. Geolocation needs a secure context (localhost counts). Chrome also gives every file:// page one shared storage partition.
3. **Open DevTools from the browser menu**. The page intercepts F12, Ctrl/Cmd+Shift+I/J and right-click. That is a deterrent, not a lock.
4. **Expect a slow first load after every edit**. The texture cache is keyed on a hash of every script, so any code change repaints the ~470 canvases once. `?texcache=0` bypasses the cache and `?texcache=clear` empties it. From the console, `__campusTex.clear()`.
5. **Test both tiers**. `?gfx=low` forces the phone tier on a desktop. A viewport under 900 px switches on the phone layout and viewer-only mode (no editor).
6. **Syntax-check before you reload**. Every inline script must still parse (all 24 do today):

Script syntax check (Node 18+)

```
python3 - <<'EOF'
import re
html = open('SastraMap_Done.html', encoding='utf-8').read()
for i, b in enumerate(re.findall(r'<script>(.*?)</script>', html, re.S)):
    open('chk_%02d.js' % i, 'w').write(b)
EOF
for f in chk_*.js; do node --check "$f" > /dev/null || echo "SYNTAX ERROR in $f"; done
```

12. Recipe A — Adding a new building

A building on this map has up to six layers. You need the geometry, and a card record if people should be able to click it. The others are optional.

Layer	Where	Required?
1 Geometry	A World-Editor asset (preferred) or a host BUILDINGS polygon	Yes
2 Placement	The <code>custom</code> array in the #sathizz-map-data payload (placed with the editor)	For editor assets
3 Name sign	<code>BUILDING_LABELS</code> (63102)	Recommended
4 Card record	<code>SASTRA_BUILDINGS</code> (66190) + <code>SASTRA_CIVIL</code> (66348)	To make it clickable
5 Academic tabs	<code>campusData</code> (66108) + <code>CAMPUS_CODES</code>	Academic blocks only
6 Routing	<code>ROAD_TARGETS</code> (73305), plus <code>ROAD_NET</code> waypoints if no road reaches it	For an on-road route

Choose the World-Editor path (A1) by default

Every mesh a host `BUILDINGS` entry creates is counted by the `ENV_ITEM` numbering (§4.5). Appending a host building shifts the index of every mesh traversed after it. Those indices are the ones the shipped layout uses to hide 404 props and move host objects. Editor placements are added after the numbering is fixed, so they are safe. That is why 290 of the campus's objects ship this way.

Step A1.1 — Write and register a builder

Follow an existing asset module, for example the "GREEN BUNNIES and the CAR PARKING SHED" block (about line 50282). Build from shared, cached materials: one material per "bin". Merge the geometry per material ("flush the bins"). Paint textures with the canvas factories, which go through `campusTexPaint`, so the texture cache covers them. Return a `THREE.Group`. Then register it:

Registry injection (pattern used by every `WE_PROPS` asset)

```
function createNewBlock() {
  var g = new THREE.Group();
  /* ...meshes built from shared materials, merged per material... */
  return g;
}
if (typeof BUILDERS !== 'undefined') BUILDERS['new_block'] = createNewBlock;
if (typeof CATALOG !== 'undefined') CATALOG.push({
  id: 'new_block', name: 'New Research Block', icon: 'B',
  tag: 'Academic - 3 storeys', group: 'Academic', cat: 'academic',
  tags: ['academic', 'block', 'campus'],          /* pin kinds are matched on these words */
  size: [40, 12, 26], anchor: 'center',           /* real metres */
  footprint: 20, height: 6,                        /* measured local-unit AABB (1 u = 2 m) */
  builder: 'new_block', brush: false, rotFree: true
});
```

Keep the builder free of `onBeforeCompile` unless you need it: the static batcher cannot merge such meshes. Do not add lights. Lamps belong to `WE_LIGHTS`, and any other light changes every lit program key.

Step A1.2 — Place it

1. On a desktop wider than 900 px, open the World Editor sidebar, choose the Props tab, equip "New Research Block" and click to place it. Position it with the gizmo, then **Save**.
2. From the console: `WorldEditor.place('new_block', 120.4, 64.1)`, adjust with the gizmo, then `WorldEditor.save()`.
3. Or edit the payload directly with the tested round-trip tool below. It is byte-identical when you make no edit. Add a record such as `{uid: 'WE_new_block_mzz0001_0', asset: 'new_block', p:[x, 0.02, z], r: yawRadians, s:[1,1,1]}`.

payload_tool.py - decode, edit and re-encode #sathizz-map-data safely

```
import json, re, sys, urllib.parse

SRC, DST = sys.argv[1], sys.argv[2]
html = open(SRC, encoding='utf-8').read()
m = re.search(r'(<div id="sathizz-map-data"[^>]*>)(.*?)(</div>)', html, re.S)
data = json.loads(urllib.parse.unquote(m.group(2)))
# ---- edit here -----
# data['custom'].append({'uid': 'WE_new_block_mzz0001_0', 'asset': 'new_block',
#                         'p': [120.0, 0.02, 60.0], 'r': 0.0, 's': [1, 1, 1]})
# -----
enc = urllib.parse.quote(json.dumps(data, separators=(',', ':'), ensure_ascii=False),
                          safe="_.!~*'()") # == encodeURIComponent
html = html[:m.start(2)] + enc + html[m.end(2):]
open(DST, 'w', encoding='utf-8').write(html)
print('custom:', len(data['custom']), 'bytes:', len(enc))
```

Step A1.3 — Sign, card and civil rows

Code to add

```
/* BUILDING_LABELS (63102) - anchor = the placement uid (obj.name) */
{ id: 'new_block', name: 'New Research Block', anchor: 'WE_new_block_mzz0001_0' },

/* SASTRA_BUILDINGS (66190) - l: label id, a: every placement of the asset */
{ key: 'new_block', name: 'New Research Block', sub: '', labs: null, classrooms: null,
  sponsor: null, match: ['l: new_block', 'a: new_block'] },
/* SASTRA_CIVIL (66348) - null renders as "--" */
new_block: { elev: null, area: null },
```

If you do not know the uid yet, select the placement in the editor and run `labelBindSelected('new_block')` in the console, then copy the uid into the table.

Step A1.4 — Academic tabs (optional)

Add a code to `campusData` and map it in `CAMPUS_CODES`. `applyCampusData()` then generates the Branches / Classrooms / Labs / Details tabs. The Details title is the `desc` up to its first ". ". Copy the existing entries' format exactly: they contain emoji with variation selectors, curly apostrophes and Indian digit grouping.

campusData entry

```
NRB: { branches: "Materials, Energy", classes: "Ground: 101-106 | First: 201-208",
      labs: "Thin Films, Battery Testing (Cyclers, EIS)",
      desc: "New Research Block: Energy Materials. Opened in 2027 ..." },
/* CAMPUS_CODES */ NRB: 'new_block',
```

Step A1.5 — Verify

- Click the building. The card opens with the right name. `SastraCard.state()` shows `has: true` and a sensible `center`.
- Names (L key): the sign sits over the roof. Adjust `lift` or `off` if it does not.
- `renderer.info.programs.length` is unchanged (74 on desktop). `__mapFPS(true)` shows a steady 60 while you orbit.
- Save, run **Export Dev build** and open the exported file: the building is present and no DOM is duplicated.

Path A2 — Host survey polygon (only if you must)

Append an entry to `BUILDINGS` in pixel space ($px = x/0.2 + 1000$, $py = z/0.2 + 654$). Then make its meshes invisible to the `ENV_ITEM` numbering **before** `assignStaticNames()` runs. A minimal patch at the end of the `buildBuildings()` loop body:

Keeping a host building out of the ENV_ITEM count

```
/* in buildBuildings(), after buildingGroups[b.id] = grp; (line ~6506) */
if (b.navExempt) grp.traverse(function (o) { o.userData.weIndexExempt = true; });

/* the new BUILDINGS record */
{ id:'new-block', name:'New Research Block', ta:'', cat:'Research', label:false, navExempt:true,
  desc:'...', descTa:'...',
  parts:[ {poly:rect(1600,1000,1680,1060), floors:3, wall:'cream', roof:'flat'} ]},
```

Check the invariant: record the `ENV_ITEM` signature below before the change and compare it after. The two strings must be identical.

Console - ENV_ITEM signature (run on the same build before and after)

```
(function () {
  var h = 2166136261 >>> 0, n = 0;
  scene.traverse(function (o) {
    if (!/^ENV_ITEM/.test(o.name)) return;
    n++;
    for (var i = 0; i < o.name.length; i++) {
      h ^= o.name.charCodeAt(i); h = Math.imul(h, 16777619) >>> 0;
    }
  });
  return n + ' names, fnv ' + h.toString(16);
})();
```

13. Recipe B — Recalibrating X/Y/Z targets

Conventions: +X east, +Z south (north is -Z), +Y up, 1 u $\approx$ 2 m in the survey frame. Road nodes carry y = the road top (0.02 to 0.32). The route tube floats 0.6 u above it and the rings 0.12 u.

Target	Where	How to recalibrate
Route end of a building	<code>ROAD_TARGETS</code> 73305	Procedure B1 below
Main-gate route start	<code>ROAD_GATE</code> 73321	Measure the forecourt point (start) and the gate threshold; both are [x, z]
Host building transform	Payload <code>buildings{}</code>	World Editor gizmo, then Save. Written by <code>syncBuildingTransformsForSave()</code>

Target	Where	How to recalibrate
Placement transform	Payload <code>custom[]</code>	World Editor gizmo, or <code>payload_tool.py</code> for exact numbers
Name sign position	<code>BUILDING_LABELS</code> lift / off	<code>labelNudge(id, dx, dz)</code> live; copy the result into <code>off</code>
GPS mapping	<code>ANCHOR</code> 71655, <code>SUNCLOCK.trueNorthDeg</code>	Procedure B2 below
Camera spawn view	<code>resetView()</code> 10118	<code>camT</code> (4, -4, 18, dist 382, az -0.46, el 0.60). dist becomes the zoom ceiling.

B1 — Recalibrating a ROAD_TARGETS entry

1. Open the building's card. Run `SastraCard.state().center` in the console to get the box centre `[x, y, z]`. Its `x` and `z` are the new `ref`.
2. Choose `t`, the visual centre of the building's main mass. For a regular block this is the box centre. For an L-shape or a courtyard plan, use a point on the principal wing. `t[1]` is the base height (≥ 0 ; the existing values are between -0.12 and 0.02).
3. Choose `door`, the point on the road network in front of the real entrance. Hover the entrance in Top view (the Top button) and run `SastraRoute.snap(x, z)`. Use `[snap.x, snap.z]` if `d` is small. Leave `door` `null` when projecting `t` onto the network already lands on the correct frontage.
4. Edit the entry, reload and open the card. `SastraRoute.state().route` should report `calibrated: true` and `shifted: null`. Ring B should stand clear in front of the entrance, and the light column should rise from the middle of the building.
5. Run the offline validator in §14. Every door should be about 0.00 u from the network.

Keep ref in step with t

`ref` is how the module notices a building has moved. If the card's measured box centre later differs from `ref` by more than 1 u, `t` and `door` are shifted by that difference. Moving a building in the editor therefore needs no recalibration. Rebuilding its geometry does, because the box changes shape: measure `ref` again whenever you change `t`.

A new calibrated target (key = the SASTRA_BUILDINGS record key)

```
var ROAD_TARGETS = {
  ...
  /* t = [x, y, z] visual centre; door = [x, z] on the network or null; ref = [x, z] box centre */
  new_block: {code: "NRB", t: [120.40, 0.02, 64.10], door: [120.35, 58.90], ref: [120.52, 64.33]},
};
```

B2 — Recalibrating the GPS mapping

1. On site, turn on location. When the dot appears, tap **Fix** in the location chip and tap where you are actually standing. This re-anchors B.
2. Walk at least 60 m in a straight line and repeat. With two points the fit solves rotation and scale. It keeps the result only if the scale is within 0.75–1.33.
3. The fit is stored per viewer (`sastra.geo.cal.v1`). To make it permanent for everyone, replace `ANCHOR` with the surveyed coordinates of the Main Entrance. If a survey shows the plan is not north-up, also set `SUNCLOCK.trueNorthDeg`. That turns the GPS fit, the sun and the compass together.
4. Check from anywhere with `SastraGeo.simulate(lat, lon, 5)` and `SastraGeo.state().fit`. Clear the fit with `SastraGeo.resetCalibration()`.

14. Recipe C — Adding waypoints to the road graph safely

`ROAD_NET` is private to the AAA5 closure, so you change it by editing the file. It cannot be patched from the console. The grid navmesh needs no hand edits: it is derived from roads and placements. Call `__navRebuild()` after editor changes.

Graph editing rules

Append only. Node and edge indices are identities that plans, diagnostics and your notes refer to. Never delete or reorder existing entries. To retire a node, remove its edges and leave the node in place.

Put nodes on centrelines. y is the road top at that point: about 0.2–0.25 on asphalt pieces, 0.02 on concrete walks, 0.3 on some forecourts. Copy the neighbouring nodes.

Place a node at every junction and at every bend of more than about 7° (the smoother treats smaller turns as straight). Spacing is otherwise free, because densify adds a point every 10 u.

Tag each edge with its surface class: 1 asphalt, 2 concrete, 3 grey pavers, 4 red pavers, 5 connector (forecourt spur). A wrong class makes A* prefer or avoid the edge incorrectly.

Keep the graph connected. If a component is disconnected, `astar()` returns null and the card draws no route.

No zero-length edges. No nodes closer than 0.4 u to each other (smoothing drops them, but projection still uses them).

Appending two nodes and two edges (indices continue from 155)

```
var ROAD_NET = {
  nodes: [
    /* ... existing 0..154 ... */,
    [120.40, 0.20, 52.10], /* 155 new walk: junction with the SOC spur */
    [120.35, 0.20, 58.90] /* 156 new walk: in front of New Research Block */
  ],
  edges: [
    /* ... existing 171 ... */,
    [114, 155, 2], [155, 156, 2]
  ]
};
```

Validate offline before you reload. The script below is tested against this revision: it reports 155 nodes, 171 edges, no unreachable nodes, 0 errors, and the 14 targets' distances from the network (doors at 0.00 u, centroids up to 42.5 u, which is the drawn final leg).

roadnet_check.py SastraMap_Done.html - structure, connectivity and target reach

```
import math, re, sys
src = open(sys.argv[1], encoding='utf-8').read()
blk = src[src.index('var ROAD_NET = {'):src.index('var ROAD_GATE')]
num = r'(-?\d.)*'
nodes = [tuple(map(float, n)) for n in
          re.findall(r'\[%s,\s*\d*\s*\]' % (num, num, num), blk[:blk.index('edges:')])]
edges = [tuple(map(int, e)) for e in
          re.findall(r'\[(\d+),\s*(\d+),\s*(\d+)\]', blk[blk.index('edges:'):blk.index('ROAD_TARGETS')])]
tg = re.findall(r'(\w+): \{code: "(\\w+)", t: \\[[^\\]]+\\}', door: (null|\\[[^\\]]+\\)'), blk)
xz = lambda i: (nodes[i][0], nodes[i][2])
err = 0
for i, (a, b, k) in enumerate(edges):
    if not (0 <= a < len(nodes) and 0 <= b < len(nodes)):
        print('edge', i, 'index out of range'); err += 1; continue
    if a == b or math.dist(xz(a), xz(b)) < 0.05:
        print('edge', i, 'zero length'); err += 1
    if k not in (1, 2, 3, 4, 5):
        print('edge', i, 'bad class', k); err += 1
for i in range(len(nodes)):
    for j in range(i + 1, len(nodes)):
        if math.dist(xz(i), xz(j)) < 0.4:
            print('nodes', i, j, 'closer than 0.4 u')
adj = {i: set() for i in range(len(nodes))}
for a, b, k in edges:
    adj[a].add(b); adj[b].add(a)
seen, stack = {0}, [0]
while stack:
    for n in adj[stack.pop()] - seen:
        seen.add(n); stack.append(n)
iso = [i for i in range(len(nodes)) if i not in seen]
```

```
def reach(x, z):
    # distance from (x, z) to the nearest edge
    best = 1e9
    for a, b, k in edges:
        (ax, az), (bx, bz) = xz(a), xz(b)
        dx, dz = bx - ax, bz - az; L2 = dx * dx + dz * dz
        t = max(0, min(1, ((x - ax) * dx + (z - az) * dz) / L2)) if L2 else 0
        best = min(best, math.hypot(x - ax - dx * t, z - az - dz * t))
    return best
print('nodes', len(nodes), 'edges', len(edges), 'targets', len(tg))
print('unreachable from node 0:', iso or 'none')
for key, code, t, door in tg:
    x, _, z = map(float, t.split(','))
    if door != 'null':
        x, z = map(float, door.strip('[]').split(','))
    print(' %20s %-9s end %.2f u from the network' % (key, code, reach(x, z)))
print('ERRORS', err)
```

Then validate live. Every one of the 14 calibrated keys must still plan, each in under a millisecond:

Console - live route regression

```
['vidyut_vihar', 'vishwakarma_joth', 'soc', 'ltc', 'sashe', 'tdc', 'tifac_core', 'chith_vihar',
 'vidhya_vihar', 'chanakya', 'polytechnic_college', 'ask1', 'ask2', 'ask3']
.map(function (k) { var p = SastraRoute.plan(k); return [k, p ? p.length : 'NO ROUTE', p && p.ms]; });
SastraRoute.state().graph; /* {nodes: 157, edges: 173, targets: 14} after the example */
```

15. Recipe D — Testing UI changes without crashing the render loop

Why the loop is fragile. `animate()` re-queues itself on its first line. An exception therefore does not stop `requestAnimationFrame`: it skips everything after the throw, including `renderFX()`, on every frame. The result is a frozen canvas and a flood of console errors. The host functions `animate()` calls (`updateLeader`, `cocFrame`, the vehicle updates) have no try/catch. Add-on modules protect the loop by wrapping their hooks.

The safe pattern

1. Write the UI as a **new IIFE script block** placed after GLASS SETTINGS and before `#sathizz-map-data`. Do not edit `animate()` for UI work.
2. On start, remove stale copies of your own DOM ids and body classes (exports bake the DOM). Add those ids and classes to `exportHTML`'s strip lists (about lines 61706 and 61726).
3. Defer 3D and scene reads until `'campus:ready'` (or check `window.__campusBoot.done`).
4. Per-frame work goes on the chained hook, **inside try/catch**, and must not allocate.
5. Write DOM only when a value changes. Never read layout (`innerWidth`, `getBoundingClientRect`, `offsetWidth`, `getComputedStyle`) inside the hook. Cache it on `resize`.
6. After any state change that should appear on screen, call `window.__mapWake()`. The loop may be resting and drawing nothing.
7. Move the camera only by writing `camT`, then `clampTarget()` / `clampDist()`. Use `cineBegin()` / `cineCommit()` for glides. Never write `camera.position`.
8. Glass CSS: put `backdrop-filter` on leaf boxes only, switch it off under `max-width: 900px`, honour `prefers-reduced-motion`, pause animations under `html.rg-rest`, and stay inside the z-index stack (§6).

Template - a loop-safe UI module

```
<script>
/* MYUI:U1 =====
MY PANEL - what, why, how (write the banner first; the next person reads it)
===== */
(function () {
  'use strict';
  ['myCSS', 'myPanel'].forEach(function (id) { /* export safety */
    var n; while ((n = document.getElementById(id))) n.parentNode.removeChild(n);
  });
  function wake() {
    if (typeof window.__mapWake === 'function') { try { window.__mapWake(); } catch (e) {} }
  }
}
```

```

var VW = innerWidth, VH = innerHeight; /* cached, never read in the frame */
addEventListener('resize', function () { VW = innerWidth; VH = innerHeight; }, { passive: true });
var _v = new THREE.Vector3(), lastX = NaN, lastY = NaN, el = null;

function onReady() {
  el = document.createElement('div'); el.id = 'myPanel'; document.body.appendChild(el);
  var prev = window.__sastraCardFrame; /* CHAIN, never replace */
  window.__sastraCardFrame = function () {
    if (prev) prev();
    try {
      if (typeof camera === 'undefined') return;
      _v.set(0, 10, 0).project(camera); /* no allocation */
      var x = Math.round((_v.x * 0.5 + 0.5) * VW * 2) / 2; /* half-pixel grid */
      var y = Math.round((-_v.y * 0.5 + 0.5) * VH * 2) / 2;
      if (x !== lastX || y !== lastY) { /* write only on change */
        lastX = x; lastY = y; el.style.transform = 'translate3d(' + x + 'px,' + y + 'px,0)';
      }
    } catch (e) { /* a UI read-out never takes the render loop down */ }
  };
  wake();
}
if (window.__campusBoot && window.__campusBoot.done) onReady();
else document.addEventListener('campus:ready', onReady, { once: true });
})();
</script>

```

Verification checklist for every UI change

- Run the syntax check (§11). Load the page and keep the console clean for 60 s of orbiting, zooming, opening and closing cards, and switching modes (M key).
- `__mapFPS(true)`: a steady 60 while you interact, and lower rates tagged "idle" when you stop. Turn it off again, because a visible FPS read-out stops the loop from resting.
- Leave the page alone for 30 s. The loop must rest (the GPU goes idle). Any input must wake it at once.
- `__mapThermal()`: your change must not cause a step down in the DPR ladder.
- `renderer.info.programs.length` is unchanged. `renderer.info.memory` geometries and textures return to their baseline after closing whatever you opened.
- Open and close a card 20 times. In `SastraRoute.state().mem`, `matsMade` = `matsFreed` (plus grave) and `geosMade` – `geosFreed` equals the live count.
- Repeat with `?gfx=low` and at a 390 px wide viewport (phone sheet layout), and with `prefers-reduced-motion` emulated.
- Export a Dev build, open it, and confirm there are no duplicate elements and that your module starts cleanly.

16. Save, export and release

Action	Result
Save (World Editor)	<code>localStorage</code> per-file key. Five rotating backups (<code>::bak1..5</code>). Written to <code>::quarantine</code> if the load was incomplete. A beforeunload auto-save runs on the desktop editor.
Export Dev build	<code>SastraMap_Dev.html</code> : the editor mounts on desktop; the file has a new <code>buildStamp</code> .
Export Final build	<code>SastraMap_Final.html</code> with <code>data-viewer="1"</code> : no editor, no <code>localStorage</code> . Ship this one to visitors.
First open of any export	The embedded payload always wins, because no save matches its new <code>buildStamp</code> .
Recovery	<code>WorldEditor</code> exposes <code>listBackups()</code> , <code>restoreBackup(slot)</code> and <code>factoryReset()</code> (<code>factoryReset</code> wipes the saved layout and the embedded payload).

17. Console API quick reference

API	Purpose
<code>__mapFPS(on)</code> , <code>__mapThermal()</code> , <code>__mapThermalDpr(i "auto")</code>	Frame rate and thermal ladder diagnostics

API	Purpose
<code>__mapWake()</code> , <code>__mapRest(ms)</code> , <code>__mapCardRest(ms)</code>	Wake the loop; tune the rest tiers
<code>__mapShadows('live' 'frozen')</code> , <code>__mapBumpShadows()</code>	Shadow cadence; mark the world as changed
<code>__mapFxState()</code> , <code>__mapFxApply(fx)</code>	Post-FX snapshot (the payload's <code>fx</code>)
<code>__campusTex</code> , <code>__campusGfx</code> , <code>__campusBoot</code> , <code>__campusSun</code> , <code>__campusAtmos</code>	Texture cache, GFX tier, boot state, sun clock, atmosphere
<code>__campusSeed(tag)</code> , <code>__campusEntropy(on)</code>	Re-seed a phase; switch to native randomness (debug only)
<code>__campusBatch.off()/on()</code> , <code>__mapCull(run)</code>	Static batcher; frustum-cull report
<code>campusRoute(ax,az,bx,bz)</code> , <code>campusRouteBuildings(a,b)</code> , <code>__navStats()</code> , <code>__navRebuild()</code>	Grid navmesh router
<code>SastraRoute.on.set.plan.origin.reroute.snap.samples.state</code>	Road-graph router
<code>SastraGeo.start.stop.follow.simulate.toWorld.calibrate</code> , <code>.resetCalibration</code> , <code>.origin.setOrigin.state</code>	GPS and route origin
<code>SastraCard.state.selection.openRecord.close.measure</code> , <code>.recordBox.setCivil.campusData ...</code>	Building card
<code>SastraPOI</code> , <code>SastraWeather</code> , <code>SastraSettings</code> , <code>SastraTags</code> , <code>SastraTip</code>	Pins, weather, settings panel, name tags, tooltip
<code>labelAudit()</code> , <code>labelBind(id,key)</code> , <code>labelBindSelected(id)</code> , <code>labelNudge(id,dx,dz)</code>	Name-sign binding and placement
<code>WorldEditor.ED.WEC.place.save.exportHTML.listBackups</code> , <code>.restoreBackup.loadReport.factoryReset</code>	Editor state, placement, save, export, backups

Part III — Future AI Agent Rules of Engagement

This part is normative. **MUST**, **MUST NOT** and **SHOULD** carry their RFC 2119 meanings. An AI agent (LLM, coding assistant or autonomous tool) that cannot meet a **MUST** stops and reports the conflict. It does not improvise around it. Human reviewers can use the same list as a review checklist.

18. Prime directives

#	Directive	Requirement
D1	Read before you write.	You MUST read the full banner comment of every block you change, and the HOTFIX / ROOTFIX / MATHFIX notes inside it. They record measured numbers and bugs that were already fixed. You MUST NOT delete or shorten them. If your change makes one out of date, add a dated note below it.
D2	Make minimal, local diffs.	You MUST NOT reformat, re-indent, rename, reorder, "modernise" (var to let, functions to arrows, adding modules or classes) or deduplicate code you were not asked to change. The file relies on declaration order, hoisting and TDZ timing across 24 scripts.
D3	Keep the map one file.	You MUST NOT add external scripts, CDNs, ES modules, fetch, XHR or WebSocket. The CSP forbids them (<code>connect-src 'none', script-src 'unsafe-inline'</code>). The one sanctioned exception is the PWA layer (Part IV), approved by the owner on 27 Sep 2026: <code>sw.js</code> , <code>manifest.webmanifest</code> and <code>icons/</code> beside the page, with <code>worker-src 'self'</code> and <code>manifest-src 'self'</code> . Any further CSP change needs the owner's explicit approval.
D4	Do not touch three.js.	You MUST NOT edit line 1958 or upgrade the library. The engine depends on r128 internals: the WebGLProgram link path, the MAX_TEXTURE_SIZE read in the constructor, <code>renderer.properties</code> programs, the TubeGeometry index layout and WebGLMultisampleRenderTarget.
D5	Handle the payload properly.	You MUST edit <code>#sathizz-map-data</code> only by decoding it, editing the JSON and re-encoding it with the encodeURIComponent character set (<code>payload_tool.py</code> , §12), or through the World Editor and an export. Never edit it with string replacement on the encoded text. Keep the migration latches (<code>forestRestored</code> , <code>recoveredV1</code> , <code>inPlaceV1</code>) and <code>buildStamp</code> .
D6	Keep public names.	You MUST keep every <code>window.Sastra*</code> , <code>window.__map*</code> , <code>window.__campus*</code> , <code>WorldEditor</code> / <code>campusBuilder</code> name and signature. Other modules, deep links and saved bookmarks use them.
D7	Treat data as data.	Card strings in <code>campusData</code> are verbatim from the client brief. They contain emoji with variation selectors, curly apostrophes and Indian digit grouping. Copy them. Never re-type or "clean" them. Only a human may change the placeholder figures (null labs, classrooms, sponsor, elev, area).
D8	Say what you did not verify.	You MUST report any check you could not run, for example because you had no browser or GPU. "Looks correct" is not verification (§22).

19. Memory-safety protocol

GPU memory is not garbage-collected. A geometry, material, texture or render target stays in VRAM until you call its `dispose()`. On iOS the whole web process is killed at about 1–1.5 GB, and this campus already uploads about 255 MB of textures (§9.2). Route updates are the most frequent allocation on the page: a moving GPS dot replans every 650 ms. So these rules are strict.

#	Level	Rule
M1	MUST	Call <code>geometry.dispose()</code> on every geometry you create per route or per update when you replace or remove it, in the same tick as the replacement. Reference implementations: <code>nmDrawRoute</code> (the old tube geometry is disposed before the new one is attached) and <code>retube()</code> 74085 (four old geometries disposed, four new ones attached to the same meshes).

#	Level	Rule
M2	MUST	Call <code>material.dispose()</code> on every material your module created for a route when that route is destroyed, but only through the safe path. Remove the object from the scene first. Then dispose the material if <code>matSafe(m)</code> is true (every program it holds has another user). Otherwise push it to GRAVE and free it on a later frame with <code>flushGrave()</code> . Reference: <code>destroyPath()</code> 73971.
M3	MUST NOT	Dispose shared resources: <code>texCache / TEX.*</code> textures, <code>wallMats / roofMats</code> , <code>NMESH.mat</code> , the route's persistent MAT set (rings, beam, head), the KEEPERS/PARK materials, the World-Editor palette materials ("one cached material per bin"), and the FX render targets (only <code>sizeFX / qApplyStep</code> may touch those). Disposing a shared material turns meshes black or forces a relink hitch on the next draw.
M4	MUST	Remove before you dispose. Call <code>parent.remove(obj)</code> before any <code>dispose()</code> , so no compile or pre-warm pass meets a disposed material. After disposal, swap a placeholder geometry and material onto detached meshes that another list may still hold (as <code>destroyPath</code> does with PH and PARK).
M5	MUST	Dispose textures and render targets you create (<code>texture.dispose()</code> , <code>rt.dispose()</code>) and drop every reference to their canvases. A <code>CanvasTexture</code> keeps its canvas alive until then.
M6	MUST	Keep the scene's node count constant while the user interacts. Pool objects, or keep a same-shaped SLOT in place (AAA5:R9, the AAA6 pin pool). The host counts nodes every 60 drawn frames. Any change calls <code>statReset()</code> , which thaws about 7,000 matrices, redraws the shadow maps and re-probes for 2.6 s.
M7	MUST	Allocate nothing in per-frame code. Hoist <code>Vector3 / Matrix4 / Box3</code> scratch objects to module scope. No <code>new</code> , array or object literals, closures, <code>.map / .filter / .concat</code> , or string building unless the result changed.
M8	MUST	Register each listener and timer once and remove it on teardown. Use <code>{once: true}</code> for <code>'campus:ready'</code> . Never add a document-level <code>touchmove</code> listener: it breaks compositor scrolling in the panels.
M9	MUST	Make leaks measurable. Keep made/freed counters, as <code>SastraRoute.state().mem</code> does, and show that after 20 open/close cycles <code>matsMade == matsFreed</code> (plus grave) and <code>geosMade - geosFreed == live</code> , and that <code>renderer.info.memory</code> is back at its baseline.

Reference pattern (mirrors `nmDrawRoute` and `destroyPath`)

```

/* Replacing a route's tube: the geometry is per-route, the material is shared. */
function replaceTube(mesh, curve, segments) {
  var old = mesh.geometry;
  mesh.geometry = new THREE.TubeGeometry(curve, segments, 0.42, 8, false);
  old.dispose(); /* M1 - always, same tick */
  /* mesh.material is NMESH.mat (shared): do NOT dispose it here - M3 */
  window.__mapWake(); /* the loop may be resting */
}

/* Tearing down a route bundle this module owns. */
function destroyBundle(B) {
  if (B.grp.parent) B.grp.parent.remove(B.grp); /* M4 - out of the scene first */
  for (var k in B.geos) B.geos[k].dispose(); /* M1 */
  for (var j in B.mats) { /* M2 - only materials we made */
    if (matSafe(B.mats[j])) B.mats[j].dispose(); else GRAVE.push(B.mats[j]);
  }
  B.geos = {}; B.mats = {}; /* drop references */
  if (0.slot && !0.slot.parent) 0.grp.add(0.slot); /* M6 - node count unchanged */
}

```

20. Performance danger zones

The sections below are where a small edit can break 60 fps, the thermal budget or the saved layout. Line ranges are for this revision. Find each zone again by its banner title or function name. Any change in these zones needs the full verification in §22, with the before and after figures reported.

#	Lines	Zone	Edits that break it
Z1	11849–12184	<code>animate()</code> — LOOP / CADENCE DECISION	Adding work, allocating, reading layout, or reordering steps. <code>tgCommit()</code> must stay directly before <code>renderFX()</code> . <code>statTick()</code> stays last before the draw. The leader runs before the chips. The early returns (hidden tab, boot hold, RELINK, rest) must keep setting <code>t0 = now</code> .
Z2	3280–3680	THERMAL BUDGET, <code>MAX_DPR</code> , <code>TG</code> , <code>GFX TIER</code> , <code>RG</code>	Raising <code>MAX_DPR</code> , <code>capFPS</code> , <code>dtMax</code> or <code>dtStep</code> . Removing the rest tiers or <code>cardRestAfter</code> . Feeding the quality ladder on idle frames. Putting tier logic in an IIFE above <code>const GFX</code> (TDZ error at parse time).

#	Lines	Zone	Edits that break it
Z3	3681–4217	SHADOW REFRESH, STATIC MATRIX FREEZE, SUBTREE SHORT-CUT	Setting <code>shadowMap.needsUpdate = true</code> or calling <code>bumpShadows()</code> every frame. Forcing <code>matrixAutoUpdate</code> on. Animating an object without <code>weNoFreeze</code> : it freezes, or the audit thaws it late.
Z4	4218–4532	SHADOW FREEZE (SHF)	Adding moving shadow casters without <code>weNoFreeze</code> , which leaves ghost shadows on the road. Toggling <code>castShadow</code> per frame changes the caster signature and redraws the 4096² map.
Z5	4815–5342	PARALLEL SHADER LINK, RUNTIME RELINK, EARLY LINK	Any runtime change to a program key (§5.4). Adding lights outside the <code>WE_LIGHTS</code> budget. Every new key is a 100 ms-plus main-thread stall when first drawn.
Z6	5721–5812	<code>init()</code> , <code>campusHostPhases()</code> , <code>linearizeMaterials()</code>	Removing the fog, turning on <code>shadowMap.autoUpdate</code> or the stencil buffer, adding host meshes (<code>ENV_ITEM</code>), or running <code>linearizeMaterials</code> twice (colours darken twice).
Z7	2165–2256	DETERMINISTIC WORLD SEED	Drawing from <code>Math.random</code> in new code during boot, or reordering boot-time draws. Every later procedural result changes: forest, uuids, textures, and the texture cache is invalidated.
Z8	2373; 60392	<code>BOUNDARY</code> (13 vertices), <code>BUILDINGS</code> , <code>assignStaticNames()</code>	Inserting boundary vertices or host meshes, or changing traversal order. The saved layout then hides and moves the wrong objects.
Z9	7931–9460	NAVMESH: <code>NMESH</code> , <code>nmBuild</code> , <code>nmAStar</code> , <code>nmDrawRoute</code>	Shrinking <code>CELL</code> (the grid grows with the square), raising <code>MAX_EXPAND</code> , raising <code>H_SCALE</code> above <code>C_MIN</code> (the search stops being optimal), allocating per search, building the grid on the boot path, or routing every frame.
Z10	9778–9940	POST-PROCESSING: <code>initFX</code> , <code>sizeFX</code> , <code>renderFX</code>	Adding full-screen passes, calling <code>makeRT</code> or <code>dispose</code> inside the frame, or reallocating targets that <code>setSize</code> already reuses.
Z11	61828–62350	STATIC BATCHER / layer 13	Using layer 13 for anything else. Giving placed assets <code>onBeforeCompile</code> or non-Float32 attributes (they drop out of batching, adding draw calls). Moving parked originals on their own.
Z12	73072–74477	AAA5 ROAD ROUTE bundle	Changing route material flags (map, <code>vertexColors</code> , <code>toneMapped</code> , side, fog), unpooling the <code>SLOT</code> , exceeding the <code>tubFor()</code> segment budget, or re-planning more often than <code>REROUTE_MS</code> .
Z13	76401–77971	AAA6 pins (sprite pool)	Growing the pool at runtime, or giving pins a new <code>SpriteMaterial</code> key. The shipped key is shared with the 20 lamp-glow sprites.
Z14	1790–1810 + module CSS	Glass CSS (62 backdrop-filter rules)	Adding backdrop-filter to large or animated elements on phones, or animating opacity on the ancestor of a backdrop-filter box. Either makes the compositor re-blur every frame.
Z15	73–711; 2744	TEXTURE CACHE, <code>mkTex</code>	Painting with web fonts before they load, reading a canvas back after its painter returns, or bypassing <code>campusTexPaint</code> . Each can store wrong pixels or lose the cache.
Z16	10093–10160	<code>CAMLIM</code> , <code>clampDist</code> , <code>clampTarget</code>	Removing the anti-void leash or the spawn-distance zoom ceiling. The user can then fly into empty ocean.

21. Scene-graph and runtime invariants

#	Invariant
I1	<code>ENV_ITEM</code> numbering is byte-identical before and after your change (signature snippet, §12).
I2	The scene node count does not change during interaction (pool or <code>SLOT</code>).
I3	Runtime objects (<code>Object3D</code> , <code>Material</code> , <code>Geometry</code> : each takes a <code>UUID</code> from <code>Math.random</code>) are built under the native generator, after <code>'campus:ready'</code> , inside a <code>withNativeRandom(fn)</code> helper (each add-on module defines its own four-line copy). Boot-phase code stays on the seeded stream.
I4	Layer 13 is reserved for the static batcher's parked originals.
I5	<code>renderer.info.programs.length</code> does not grow (74 desktop / 70 phone). Reuse existing material keys.

#	Invariant
I6	Decorative runtime objects carry <code>aaaFx</code> , <code>weIndexExempt</code> , <code>weNoFreeze</code> (if they animate), <code>weNoCull</code> (if culling would clip them) and a no-op raycast. They cast no shadows.
I7	Per-frame hooks chain the previous hook and run inside try/catch. No module replaces another module's hook.
I8	Every state change that should appear on screen calls <code>__mapWake()</code> .
I9	The camera is moved only through <code>camT</code> with <code>clampTarget</code> / <code>clampDist</code> . <code>applyCam()</code> owns <code>camera.position</code> , except for Character mode's documented overwrite.
I10	Every new DOM id and body class is in the exporter's strip lists, and the module removes stale copies on start.
I11	Fog, the sun and the hemisphere light are never added or removed at runtime. Only their uniforms and properties change.
I12	Road-graph and ROAD_TARGETS edits are append-only and pass <code>roadnet_check.py</code> and the live 14-key regression (§14).

22. Change protocol for AI agents

1. **Locate.** Find the banner block(s) by title. Read each one from the banner to its closing `}}()`; or function end.
2. **Plan.** Before editing, list the directives (§18), memory rules (§19), danger zones (§20) and invariants (§21) the change touches.
3. **Edit minimally.** Tag new code with a unique greppable marker in the file's own style (`AAA5:R3`, `AAA6:08`), for example `AI1:U1`, and write a banner comment for any new block.
4. **Verify statically.** Run the syntax check (§11). If you touched the payload or the road graph, run the payload round-trip and `roadnet_check.py`.
5. **Verify at runtime** (a browser is required). Keep the console clean. Measure `__mapFPS` before and after. Confirm `renderer.info.programs.length` is unchanged, the memory counters balance, the `ENV_ITEM` signature is unchanged, the loop still rests after 30 s, `?gfx=low` and phone width work, and an Export Dev build round-trip succeeds.
6. **Report.** State what changed (line ranges and markers), which invariants you checked and how, the measurements, and anything you could not verify.

Stop conditions: report instead of proceeding when

the change would add a network call or an external asset; the program count would grow; the `ENV_ITEM` signature would change; a road-graph edit would delete or reorder indices; a request asks you to fill in real-world figures (labs, areas, coordinates) you cannot source; or you cannot run the runtime checks and the change touches zones Z1–Z6.

Part IV — PWA Offline Shell & Campus Analytics

The installable, offline-capable build and its privacy-first telemetry. The kit lives in the repository under `pwa/` and is applied to the map by `pwa/build_pwa.py`. It adds two marked script blocks (`PWA1`, `TEL1`), one head block and three companion files.

23. What the build changes

Edit	Where	Content
CSP	Line 14	<code>worker-src 'none'</code> becomes <code>worker-src 'self'</code> ; <code>manifest-src 'self'</code> is added. <code>connect-src 'none'</code> is unchanged, so nothing can leave the device.
Head block	After the viewport meta	<code>theme-color #000000</code> , <code>mobile-web-app-capable</code> , <code>apple-mobile-web-app-capable</code> , status bar <code>black</code> , <code>apple-mobile-web-app-title</code> , manifest link, favicon, apple-touch-icon.
TEL1 script	Before <code>#sathizz-map-data</code>	<code>CampusAnalytics</code> and its hooks. It observes and never edits host code.
PWA1 script	After TEL1	Service-worker registration after boot, offline/online and update notices through the host <code>toast()</code> , and the install chip (<code>#pwaCSS</code> , <code>#pwaChip</code>).
Exporter	<code>exportHTML()</code> strip list	<code>#pwaCSS</code> , <code>#pwaChip</code> are added so exports never bake the chip.
Companions	Next to the page	<code>sw.js</code> (VERSION = buildStamp + sha256 of the page and the worker), <code>manifest.webmanifest</code> , <code>icons/</code> (192, 512, maskable 512, apple-touch 180, favicon 32).

The scripts come after the host engine, so the texture cache's host-painter key (a hash of every script up to the host script) is unchanged: returning visitors keep their cached textures. Why not a single file? A service worker must be a same-origin http(s) script, so `register()` rejects data: and blob: URLs. Offline support therefore needs a real `sw.js`, served over HTTPS or on localhost.

24. The offline model

The map is one document. `three.js`, the engine, the CSS, the JSON layout, the card dictionary, `ROAD_NET` and both A* routers are all inline, so caching one response makes the whole map work offline.

Request	Strategy	Notes
Navigation to the page	Cache first	Every <code>?query</code> and <code>#hash</code> gets the one cached shell. The network is used only while nothing is cached; with no cache and no network, a small offline page explains what to do.
Manifest, icons, other same-origin files	Cache first	Pre-cached at install (best effort); other files are kept after their first fetch.
Google Fonts CSS	Stale-while-revalidate	Fetches from the worker at install with the same user agent; every woff2 it names is cached.
Google Fonts files	Cache first	Cached as CORS responses; the font face falls back to system fonts if never fetched.

- **Install.** The shell is fetched with `cache: 'no-cache'`, a conditional GET. A 304 reuses the HTTP-cached copy, so the first visit does not download 4.5 MB twice. The install fails, and is retried later, unless the response is a 200 same-origin HTML page.
- **Update.** The page registers with `updateViaCache: 'none'`, so each launch re-checks `sw.js`. A new page means a byte-different `sw.js`. The new worker pre-caches, calls `skipWaiting`, deletes the old shell cache and messages the page, which shows "Map updated — reopen it to use the new version". Swapping workers mid-session is safe

because the app has no lazy chunks.

- **Never blocks the boot.** `register()` is armed on `campus:ready` and then waits 4 s plus an idle slot. There is a 90 s fallback after `load` if the boot never releases. Neither block has per-frame code.
- **Storage.** An installed copy calls `navigator.storage.persist()` to protect the worker cache and the texture cache from eviction.

25. CampusAnalytics

Hook	Source it observes	Data
<code>onSearchQuery(term, meta)</code>	Enter or result click in <code>#axSearchIn</code> (capture phase); a query that settles 1.5 s with no results	term, results, chosen, chosenKey, via
<code>onRouteGenerated(start, end, metres, meta)</code>	'sastra:route' (once per trip, not per GPS re-plan) and the Route button via the wrapped global <code>nmRouteBuildings</code>	start, end, metres, minutes, router, startKind, startVertex, endKey, path (road-graph node ids)
<code>onPOIToggled(type, on)</code>	aria-pressed on <code>#pxFood ... #pxPark</code> (or <code>#euFood / #euWC</code>)	poi, state
<code>onBuildingViewed(key, name)</code>	'sastra:card' open	key, name
<code>onAppLaunch()</code>	once, at <code>campus:ready</code>	bootMs; ctx.display, ctx.online

Each event is `{v, type, t, seq, sid, ctx, data}`. The console sink prints a readable line and then the JSON, for example:

Console output captured in the verification run (JSON wrapped for print)

```
[TELEMETRY] Route generated: Main Gate to Library SASTRA - 442m
{"v":1,"type":"route_generated","t":"2026-09-27T08:30:00Z","seq":4,"sid":"36515ebaf4ae3c51",
  "ctx":{"display":"browser","online":true,"tier":"high","lang":"en-US","build":"Bmuds6ios-9kbp"},
  "data":{"start":"Main Gate","end":"Library SASTRA","metres":442,"minutes":6,"router":"road",
    "startKind":"gate","startVertex":null,"endKey":"library","path":[1,2,3,49,94,96,112,152,113,117],
    "partial":false}}
```

```
[TELEMETRY] Search: "swimming pool" - no results
{"v":1,"type":"search_query","t":"2026-09-27T08:30:00Z","seq":5,"sid":"36515ebaf4ae3c51",
  "ctx":{"display":"browser","online":true,"tier":"high","lang":"en-US","build":"Bmuds6ios-9kbp"},
  "data":{"term":"swimming pool","results":0,"chosen":null,"chosenKey":null,"via":"typing"}}
```

```
[TELEMETRY] POI toggled: food ON
{"v":1,"type":"poi_toggled","t":"2026-09-27T08:30:00Z","seq":6,"sid":"36515ebaf4ae3c51",
  "ctx":{"display":"browser","online":true,"tier":"high","lang":"en-US","build":"Bmuds6ios-9kbp"},
  "data":{"poi":"food","state":"on"}}
```

- **Privacy.** No cookies and no persistent identifier. The session id comes from `crypto.getRandomValues` and lives for one page load (`Math.random` is the seeded world stream and would give every visitor the same id). Timestamps are rounded to the minute. No user agent, IP or raw coordinates are recorded: a GPS start is "your location" plus its snapped road vertex. Search terms are capped at 48 characters, with e-mail addresses and long numbers redacted. GPC, DNT, `optOut()` or `localStorage['sastra.telemetry']='off'` stop recording.
- **Backend, later.** (1) Add exactly one origin to `connect-src`. (2) Register a batching sink: a `localStorage` queue flushed with `sendBeacon` when the browser goes online or the tab is hidden. (3) Receive the events on a SASTRA endpoint or a Firebase HTTPS Cloud Function; do not add the Firebase JS SDK to the page. (4) Aggregate nightly. [pwa/README.md](#) §6 has the code.
- **Heatmaps for the VC.** Every road route carries `path`, its `ROAD_NET` node chain. Counting consecutive node pairs per hour gives foot traffic per road segment, built from routes people chose rather than from tracking anyone. `building_viewed` gives destination interest, and zero-result searches give unmet demand. Release only aggregates with at least 10 sessions per cell, keep raw events at most 30 days, and follow the DPDP Act 2023 notice and consent duties.

26. Verification

Headless Chromium 141 (Playwright), WebGL on SwiftShader, the build served by `python3 -m http.server` on localhost. The dead-zone step **kills the web server** before reloading. Full logs: [pwa/TEST_RESULTS.md](#).

Check	Result
Boot time, original vs PWA build	campus:ready at 81.3 s (original) vs 79.9 / 81.7 / 84.5 s (three PWA runs). No measurable delay; software WebGL.
Service worker registration	11.5 s, 20.9 s and 12.6 s after campus:ready (armed on ready + 4 s + idle slot).
Offline cache	sastramap-shell-<VERSION> holds index.html (engine, three.js, CSS, JSON, road graph); sastramap-fonts-v1 created.
Installability (Chromium, normal profile)	Page.getInstallabilityErrors: none . Manifest errors: none.
Telemetry hooks	All fired with the expected JSON: app launch, 2 searches (one with no results), building views, 3 POI toggles, road route and grid route.
Dead zone (web server killed)	Reload returned 200 from the service worker ; the full scene booted again (6,283 meshes).
A* offline	Grid router answered (3 waypoints); road graph planned SOC: 376.9 u, 12 waypoints, 0.6 ms.
Errors	No page exceptions. Only failed requests: Google Fonts (unreachable in the sandbox) and images/velaa_logo.jpg, which the original also requests and the host does not ship (a painted sign replaces it).
Not verified here	A real Android install prompt, iOS Safari Add to Home Screen, standalone display on a phone, and GPS on campus. These need physical devices.

27. Rules for agents working on the PWA layer

#	Rule
P1	Ship <code>index.html</code> and its <code>sw.js</code> together, and always through <code>build_pwa.py</code> . Never hand-edit VERSION.
P2	Keep <code>connect-src 'none'</code> until the owner approves a telemetry endpoint. Then add exactly that one origin.
P3	Telemetry observes, it never edits the host. Use events, <code>aria-*</code> observers or a wrapper of a global function that returns the original result. Every hook sits inside try/catch.
P4	Identifiers come from <code>crypto.getRandomValues</code> , never from <code>Math.random</code> . Record no raw coordinates, user agent or persistent id.
P5	Never register the worker, or start network work, before <code>campus:ready</code> . Never add per-frame code to PWA1 or TEL1.
P6	New DOM ids go on the exporter strip list, and the module removes stale copies on start (§15).
P7	Do not widen the worker's scope or cache third-party origins beyond Google Fonts. Keep other HTML out of the map's directory.
P8	Re-run <code>pwa/test/pwa_e2e.js</code> after any change to PWA1, TEL1 or <code>sw.js</code> , including the dead-zone step.

Appendix A — URL flags for testing

Flag	Effect
?gfx=low high	Force the GFX tier
?dpr=n	Pin the pixel ratio (MAX_DPR still applies)
?tg=0	Disable the adaptive-retina ladder
?fps=n	Engaged frame cap (0 = uncapped; default 60)
?msaa=0 1	Scene-target MSAA off / forced to maximum
?shadows=live	Old live shadow cadence instead of the freeze
?texcache=0 clear	Bypass / empty the IndexedDB texture cache
?fog=0	Park the fog ramp
?sun=0	Disable the sun clock
?hour=9.5	Pin the sun clock to an hour
?clock=0	Disable the daylight tint

Flag	Effect
?reveal=...	Progressive reveal control
?route=0 1	Road route off / on for this visit
?geo=lat,lon off	Move the GPS anchor / never auto-start GPS
?pins=food,wc 0	Turn pin kinds on / skip the pin module
?weather=off motes rain	Weather for this visit
?menu=0	Skip the glass settings module
?dof=..., ?radar=..., ?glow=...	Cinematic module switches
?poi, ?sec, ?admin, ?access, ?hc, ?link	Visitor/Access module parameters (see their banners)
?haptic, ?fat, ?tip	Touch feedback, fat-finger picking, tooltip
?sw=0 (PWA build)	Skip service-worker registration for this visit
?telemetry=0 1 (PWA build)	CampusAnalytics off / forced on for this visit

Appendix B — Glossary

Term	Meaning
Banner	The <code>/* ==== TITLE ==== */</code> comment that opens a subsystem. It is the specification.
Host	The main engine script (2165–29085) and the objects it builds from the survey tables.
World Editor (WE)	The in-page editor (29113–63000) that places catalog assets and saves the layout.
Placement	One editor object: <code>{uid, asset, p, r, s}</code> in the payload's <code>custom</code> array.
Payload	The URL-encoded JSON in <code>#sathizz-map-data</code> (line 78757).
buildStamp	The lineage id written by each export. It decides whether localStorage or the payload wins.
ENV_ITEM_n	The traversal-order name of an unnamed host mesh, used by the saved layout.
Program key	The set of material and renderer features three.js hashes into one GPU shader program.
Engaged / rest	Governor states: the user is interacting / nothing is drawn at all.
Rung	One pixel-ratio step on the adaptive-retina ladder [2, 1.5, 1.25].
Door	The road-network point in front of a building's real entrance (ROAD_TARGETS).
KEEPERS / GRAVE / SLOT	Route-module devices: keep shared programs alive, defer unsafe disposes, keep the node count constant.
Layer 13	The invisible layer that holds the batched originals.